

UNIVERSITÉ ACADÉMIQUE —/ MASTER 2

Filière : Informatique — MSc Artificial Intelligence & Digital Transformation

MÉMOIRE DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du diplôme de Master 2

Conception et développement d'une plateforme intelligente D'automatisation du reporting analytique des applications mobiles



Réalisé par : Bend dhif Balsam

Encadré par : Zargayouna Mahdi

Année universitaire : 2025 – 2026

Déclaration

Je déclare sur l'honneur que le présent mémoire est le fruit d'un travail personnel réalisé dans le cadre académique du Projet de Fin d'Études. Je déclare qu'il n'a pas été soumis, en tout ou en partie, dans le cadre d'une autre évaluation académique.

Je déclare également que ce travail est entièrement académique : il ne fait référence à aucune entreprise réelle, aucun client, aucun stage ni aucun contexte professionnel identifié. Toutes les données présentées dans le cadre expérimental sont simulées ou anonymisées à des fins strictement pédagogiques et scientifiques.

Je certifie que toutes les sources documentaires utilisées pour la rédaction de ce mémoire ont été correctement citées et référencées dans la bibliographie, conformément aux exigences d'intégrité scientifique et académique.

Remerciements

Au terme de ce travail de fin d'études, je tiens à exprimer ma sincère reconnaissance à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce mémoire.

Je remercie tout d'abord mon encadrant académique pour la qualité de son suivi, la pertinence de ses conseils méthodologiques et sa disponibilité tout au long de ce projet de recherche appliquée. Ses orientations concernant les choix architecturaux et la rigueur scientifique attendue ont grandement enrichi ce travail.

Mes remerciements s'adressent également au corps professoral et administratif de l'établissement, pour la qualité de la formation dispensée durant ce cursus, qui m'a permis d'acquérir les compétences théoriques et pratiques nécessaires à la conduite de ce projet.

Je souhaite aussi remercier les membres du jury pour l'intérêt qu'ils porteront à ce travail ainsi que pour le temps consacré à son évaluation. Enfin, une pensée particulière va à mes proches, pour leur soutien moral constant durant cette période de travail intense.

Dédicaces

Je dédie ce modeste travail :

À mes parents, pour leur soutien indéfectible, leurs encouragements et leurs sacrifices tout au long de mon parcours académique ;

À ma famille, pour sa patience et sa confiance ;

À mes amis et collègues de promotion, pour les échanges enrichissants qui ont accompagné ce travail ;

À toute personne animée par la curiosité scientifique et l'envie d'innover dans le domaine des données et de l'intelligence artificielle.

Résumé

Le suivi de la performance des applications mobiles constitue aujourd'hui un enjeu stratégique majeur pour les équipes produit, marketing et techniques. Les plateformes de distribution telles que Google Play et l'App Store mettent à disposition une quantité importante de données brutes — installations, désinstallations, notes, avis, revenus, plantages — mais ne proposent que des outils d'analyse limités, cloisonnés par plateforme, et ne permettant pas une automatisation de bout en bout du cycle de reporting. Dans la pratique, la production de rapports analytiques reste très largement manuelle, chronophage, sujette à l'erreur humaine et peu réactive face à des anomalies critiques telles qu'un pic de désinstallations ou une chute soudaine de la note moyenne.

Ce mémoire présente la conception et le développement d'une plateforme intelligente d'automatisation du reporting analytique des applications mobiles, combinant les principes du Data Engineering, les architectures Cloud modernes et les capacités des grands modèles de langage (Large Language Models, LLM) relevant de l'intelligence artificielle générative. La solution proposée met en œuvre un pipeline complet de type ELT (Extract – Load – Transform) orchestré par le moteur d'automatisation n8n, permettant la collecte automatisée des données depuis les API officielles de Google Play Console et d'App Store Connect, leur nettoyage et leur transformation, le calcul automatique d'indicateurs clés de performance (KPIs), leur stockage dans une base de données relationnelle hébergée sur la plateforme Supabase/PostgreSQL, ainsi que la génération automatique de rapports au format PDF, de présentations PowerPoint et d'emails de synthèse. Un module d'intelligence artificielle générative, s'appuyant sur l'API d'OpenAI, est intégré au pipeline afin de produire des commentaires analytiques contextualisés, de détecter des anomalies et de formuler des recommandations actionnables à destination des équipes métier.

L'architecture proposée est détaillée selon une approche de conception logicielle rigoureuse, incluant la modélisation UML des besoins et des composants, la conception d'un schéma de données de constellation adapté aux besoins de Business Intelligence, ainsi qu'une architecture Cloud modulaire et évolutive. Les résultats obtenus démontrent une réduction significative du temps de production des rapports, une amélioration de la fiabilité des données ainsi qu'une valeur ajoutée analytique apportée par la couche d'intelligence artificielle générative. Ce travail ouvre par ailleurs des perspectives d'amélioration, notamment en

matière de passage à l'échelle, de gouvernance des données et d'enrichissement des modèles prédictifs.

Mots-clés : Data Engineering, Intelligence Artificielle Générative, Cloud Computing, Automatisation, Business Intelligence, App Store Optimization, Large Language Model, n8n, ELT, Reporting analytique.

Abstract

Monitoring the performance of mobile applications has become a major strategic concern for product, marketing and engineering teams. Distribution platforms such as Google Play and the App Store expose a large volume of raw data — installs, uninstalls, ratings, reviews, revenue, crashes — yet only provide limited, platform-specific analytical tools that do not allow for an end-to-end automation of the reporting cycle. In practice, the production of analytical reports remains largely manual, time-consuming, error-prone, and poorly responsive to critical anomalies such as a spike in uninstalls or a sudden drop in average rating.

This thesis presents the design and development of an intelligent platform for automating analytical reporting of mobile applications, combining Data Engineering principles, modern Cloud architectures, and the capabilities of Large Language Models (LLMs) as part of generative artificial intelligence. The proposed solution implements a complete ELT (Extract – Load – Transform) pipeline orchestrated by the n8n automation engine, enabling automated data collection from the official Google Play Console and App Store Connect APIs, data cleaning and transformation, automatic computation of key performance indicators (KPIs), storage in a relational database hosted on the Supabase/PostgreSQL platform, as well as the automatic generation of PDF reports, PowerPoint presentations and summary emails. A generative artificial intelligence module, relying on the OpenAI API, is integrated into the pipeline to produce contextualized analytical commentary, detect anomalies, and formulate actionable recommendations for business teams.

The proposed architecture is detailed through a rigorous software design approach, including UML modeling of requirements and components, the design of a constellation schema data model adapted to Business Intelligence needs, and a modular, scalable Cloud architecture. The results obtained demonstrate a significant reduction in report production time, improved data reliability, and analytical value added by the generative AI layer. This work also opens perspectives for improvement, particularly regarding scalability, data governance, and the enrichment of predictive models.

Keywords: Data Engineering, Generative Artificial Intelligence, Cloud Computing, Automation, Business Intelligence, App Store Optimization, Large Language Model, n8n, ELT, Analytical Reporting.

Mots-clés

Data Engineering

Intelligence Artificielle Générative

Cloud Computing

Business Intelligence

Large Language Model

App Store Optimization

Pipeline ELT

Automatisation des workflows

n8n

Supabase / PostgreSQL

Star Schema

OAuth 2.0

Prompt Engineering

Google Play Developer Reporting API

App Store Connect API

Liste des acronymes

Acronyme	Signification
API	Application Programming Interface
ASO	App Store Optimization
BI	Business Intelligence
BPMN	Business Process Model and Notation
CI/CD	Continuous Integration / Continuous Deployment
CSV	Comma-Separated Values
ELT	Extract – Load – Transform
ETL	Extract – Transform – Load
GA4	Google Analytics 4
GPT	Generative Pre-trained Transformer
HTML	HyperText Markup Language
IA	Intelligence Artificielle
IaaS	Infrastructure as a Service
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
OAuth	Open Authorization
PaaS	Platform as a Service
PDF	Portable Document Format
REST	Representational State Transfer
RGPD	Règlement Général sur la Protection des Données
RLS	Row Level Security

SaaS	Software as a Service
SDK	Software Development Kit
SGBD	Système de Gestion de Base de Données
SLA	Service Level Agreement
SQL	Structured Query Language
UML	Unified Modeling Language
VM	Virtual Machine

Liste des figures

Figure 1. Cycle de vie de l'optimisation ASO (mots-clés, conversion, rétention, itération) ...	17
Figure 2. Les trois modèles de service Cloud (IaaS, PaaS, SaaS) et répartition des responsabilités.....	19
Figure 3. Comparaison schématique des flux ETL et ELT	22
Figure 4. Architecture générale d'intégration d'un LLM dans un pipeline analytique (données → prompt structuré → réponse → intégration au rapport)	23
Figure 1. Représentation du processus manuel actuel de reporting analytique mobile.....	27
Figure 2. Diagramme de contexte du système.....	29
Figure 3. Diagramme de cas d'utilisation UML de la plateforme.....	29
Figure 4. Diagramme BPMN du processus automatisé de reporting.....	33
Figure 1. Comparaison schématique des flux d'authentification OAuth 2.0 (Google) et JWT (Apple)	39
Figure 1. Architecture globale de la plateforme proposée (vue en couches).....	46
Figure 2. Diagramme de composants de l'architecture logique	47
Figure 3. Diagramme de classes UML du domaine métier.....	48
Figure 4. Diagramme de séquence UML — Génération automatisée d'un rapport.....	49
Figure 5. Diagramme d'activité UML du pipeline complet.....	50
Figure 6. Diagramme de déploiement UML de la plateforme proposée	51
Figure 7. Architecture Cloud globale de la plateforme proposée	52
Figure 8. Pipeline ELT détaillé de la plateforme proposée.....	53
Figure 9. Pipeline d'intégration du module LLM	54
Figure 10. Vue d'ensemble du workflow n8n orchestrant le pipeline complet.....	55
Figure 11. Architecture des API consommées et exposées par la plateforme	56
Figure 12. Vue d'ensemble de l'architecture de sécurité de la plateforme proposée.....	57

Figure 1. Modèle Entité-Association (MCD) de la plateforme proposée	60
Figure 2. Schéma en étoile (Star Schema) de l'entrepôt de données	61
Figure 1. Aperçu du gabarit HTML du rapport de performance	71
Figure 2. Vue détaillée du workflow n8n implémenté	74
Figure 1. Flux détaillé d'authentification OAuth 2.0 (Client Credentials) pour Google Play...	76
Figure 2. Flux détaillé de génération et d'utilisation du jeton JWT pour App Store Connect.	77
Figure 1. Pyramide de tests appliquée à la plateforme proposée	81
Figure 2. Courbe du temps d'exécution du pipeline en fonction du nombre d'applications suivies	85
Figure 1. Évolution hebdomadaire des installations — Applications Android simulées	88
Figure 3. Évolution hebdomadaire du taux de désinstallation — Applications iOS simulées	89
Figure 5. Extrait du tableau de bord de synthèse généré pour un scénario d'anomalie.....	91
Figure 6. Tableau de bord synthétique multi-applications	92

Liste des tableaux

Tableau 1. Comparaison entre Google Play Console et App Store Connect	16
Tableau 2. Comparaison entre schéma en étoile et schéma en flocon	20
Tableau 3. Comparaison entre les approches ETL et ELT	21
Tableau 4. Comparaison des solutions existantes de reporting analytique mobile	23
Tableau 1. Identification des acteurs du système	28
Tableau 2. Description du cas d'utilisation « Générer un rapport de performance »	30
Tableau 1. Comparaison des langages envisagés pour l'implémentation du pipeline	34
Tableau 2. Comparaison des systèmes de gestion de base de données envisagés	35
Tableau 3. Comparaison des plateformes Backend-as-a-Service envisagées	36
Tableau 4. Comparaison des approches de déploiement envisagées	37
Tableau 5. Comparaison des mécanismes d'authentification retenus	39
Tableau 6. Comparaison des plateformes d'orchestration de workflows envisagées	40
Tableau 7. Comparaison des approches de génération de rapports PDF	41
Tableau 8. Comparaison des canaux de diffusion envisagés	42
Tableau 9. Comparaison des modèles de langage envisagés pour le module d'analyse	42
Tableau 10. Comparaison des techniques de Prompt Engineering envisagées	43
Tableau 11. Matrice de décision technologique synthétique	44
Tableau 1. Description synthétique des classes principales du domaine	48
Tableau 2. Répartition des responsabilités par nœud physique	51
Tableau 3. Étapes du pipeline ELT et responsabilités associées	53
Tableau 1. Description de la table de faits FaitMetriqueJournaliere	61
Tableau 2. Description de la table de faits FaitAvis	62
Tableau 3. Description des tables de dimensions	62
Tableau 4. Tables opérationnelles complémentaires	63
Tableau 1. Catalogue des indicateurs de performance (KPIs) calculés	68
Tableau 2. Nœuds principaux du workflow n8n orchestrant le pipeline	73
Tableau 1. Politique de gestion des secrets de la plateforme proposée	78
Tableau 2. Analyse des menaces selon le modèle STRIDE	79
Tableau 1. Extrait des cas de tests unitaires du module de calcul des KPIs	82
Tableau 2. Extrait des cas de tests d'intégration	83
Tableau 3. Scénarios de tests de bout en bout	83
Tableau 4. Résultats de la validation qualitative des analyses générées par le LLM (échantillon de 20 scénarios)	84
Tableau 5. Résultats des tests de performance du pipeline complet	85
Tableau 1. Caractéristiques du cadre expérimental	87
Tableau 2. Comparaison synthétique des KPIs moyens observés (Android vs iOS, données simulées)	89
Tableau 3. Résultats de la détection d'anomalies sur les six cas injectés	90

Table des matières

Déclaration	2
Remerciements	3
Dédicaces.....	4
Résumé	5
Abstract.....	7
Mots-clés	8
Liste des acronymes	9
Introduction générale.....	19
Chapitre 1 — État de l'art	21
1.1 Les applications mobiles et leurs écosystèmes de distribution	21
1.1.1 Google Play Console	21
1.1.2 App Store Connect	21
1.2 App Store Optimization (ASO)	22
1.3 Business Intelligence (BI)	23
1.4 Data Engineering	24
1.5 Cloud Computing	24
1.6 Big Data.....	25
1.7 Entrepôts de données et modélisation dimensionnelle	25
1.8 Paradigmes ETL et ELT	26
1.9 Automatisation des workflows (Workflow Automation).....	28
1.10 Grands modèles de langage (LLM) et intelligence artificielle générative	28
1.10.1 Prompt Engineering.....	28
1.11 Synthèse comparative des solutions existantes	29
Chapitre 2 — Analyse des besoins	32
2.1 Présentation du problème	32

2.2 Étude de l'existant et processus manuel.....	32
2.3 Analyse critique de l'existant.....	33
2.4 Contraintes du projet	33
2.5 Acteurs du système.....	34
2.6 Diagramme de contexte	34
2.7 Cas d'utilisation (diagramme UML).....	35
2.8 Besoins fonctionnels.....	36
2.9 Besoins non fonctionnels.....	37
2.10 User Stories	37
2.11 Modélisation BPMN du processus cible	38
Chapitre 3 — Étude comparative des technologies.....	40
3.1 Langage de programmation : Python.....	40
3.2 Système de gestion de base de données : PostgreSQL.....	41
3.3 Plateforme Backend-as-a-Service : Supabase	42
3.4 Conteneurisation : Docker.....	42
3.5 Gestion de versions : Git et GitHub	43
3.6 Interfaçage avec les API : REST, OAuth 2.0 et JWT.....	43
3.7 Orchestration de workflows : n8n	45
3.8 Génération de documents : HTML, CSS et Puppeteer.....	46
3.9 Diffusion des livrables : Gmail API et Slack API.....	47
3.10 Modèles de langage : OpenAI GPT-4o et Claude (Anthropic)	47
3.11 Ingénierie de prompt (Prompt Engineering).....	48
3.12 Synthèse de la matrice de décision technologique	49
Chapitre 4 — Conception de l'architecture	51
4.1 Architecture globale de la solution développée.....	51
4.2 Architecture logique	52
4.3 Diagramme de classes	52

4.4 Diagramme de séquence : génération d'un rapport.....	54
4.5 Diagramme d'activité : pipeline complet.....	54
4.6 Architecture physique et architecture de déploiement	55
4.7 Architecture Cloud	57
4.8 Pipeline ETL/ELT détaillé	57
4.9 Pipeline d'intégration du LLM.....	59
4.10 Pipeline d'orchestration des workflows (n8n)	59
4.11 Architecture des API	60
4.12 Vue d'ensemble de l'architecture de sécurité.....	61
Chapitre 5 — Conception de la base de données	64
5.1 Démarche de modélisation retenue.....	64
5.2 Modèle Entité-Association (MCD)	64
5.3 Justification du choix du schéma en étoile	65
5.4 Modèle en étoile (Star Schema) retenu.....	65
5.5 Tables complémentaires du système	68
5.6 Relations et contraintes d'intégrité.....	68
5.7 Indexation et optimisation des performances	69
5.8 Sécurité au niveau des lignes (Row Level Security)	69
Chapitre 6 — Implémentation	71
6.1 Module de collecte des données	71
6.2 Module de nettoyage et de contrôle qualité.....	72
6.3 Module de calcul des KPIs	73
6.4 Module de détection d'anomalies	74
6.5 Module d'analyse par intelligence artificielle générative	74
6.6 Module de génération des livrables	75
6.6.1 Génération du rapport HTML et PDF	75
6.6.2 Génération de la présentation PowerPoint.....	76

6.6.3 Génération et envoi de l'email de synthèse.....	77
6.7 Orchestration globale via n8n.....	77
Chapitre 7 — Sécurité	81
7.1 Flux d'authentification OAuth 2.0 (Google Play).....	81
7.2 Flux d'authentification par jeton JWT (App Store Connect).....	82
7.3 Gestion des secrets	83
7.4 Sécurité au niveau des lignes (Row Level Security)	83
7.5 Chiffrement des communications.....	84
7.6 Analyse des menaces (Threat Modeling)	84
7.7 Analyse critique de l'architecture de sécurité	85
Chapitre 8 — Tests et validation	86
8.1 Stratégie de tests retenue	86
8.2 Tests unitaires	87
8.3 Tests d'intégration.....	87
8.4 Tests de bout en bout (End-to-End).....	88
8.5 Validation de la fiabilité des analyses générées par le LLM.....	89
8.6 Tests de performance.....	90
8.7 Analyse critique de la démarche de validation.....	90
Chapitre 9 — Résultats et discussion	92
9.1 Présentation du cadre expérimental.....	92
9.2 Analyse des résultats — Google Play (Android).....	92
9.3 Analyse des résultats — App Store (iOS)	93
9.4 Comparaison inter-plateformes	94
9.5 Détection des anomalies sur le cadre expérimental.....	94
9.6 Exemple d'analyse générée par le module d'intelligence artificielle générative	95
9.7 Tableau de bord synthétique.....	96
9.8 Discussion.....	97

9.9 Analyse critique des apports et des limites.....	98
Conclusion générale	99
Perspectives	100
Bibliographie	102
Data Engineering, Data Warehousing et Business Intelligence	102
Cloud Computing et infrastructures	103
Génie logiciel, modélisation et architecture	103
Sécurité, authentification et protocoles	104
Intelligence artificielle, grands modèles de langage et prompt engineering	105
Applications mobiles, App Store Optimization et mining d'avis	106
Tests, qualité logicielle et méthodes agiles	106
Visualisation et restitution de données	107
Documentation technique officielle.....	107

Introduction générale

Le marché des applications mobiles connaît depuis plus d'une décennie une croissance continue, portée par la démocratisation des smartphones et la diversification des usages numériques. Les deux principales plateformes de distribution, Google Play pour l'écosystème Android et l'App Store pour l'écosystème iOS, hébergent aujourd'hui plusieurs millions d'applications et génèrent des volumes considérables de données comportementales : nombre d'installations, taux de désinstallation, avis et notes des utilisateurs, revenus générés, taux de plantage, ou encore performance des campagnes d'acquisition. Ces données constituent une ressource stratégique de premier plan pour les équipes produit, marketing, techniques et direction, car elles conditionnent la capacité d'une organisation à comprendre l'usage réel de son application, à détecter rapidement des dysfonctionnements et à orienter ses décisions de manière factuelle.

Cependant, l'exploitation de ces données demeure aujourd'hui largement artisanale. Les consoles officielles (Google Play Console et App Store Connect) proposent des tableaux de bord natifs, mais ceux-ci restent cloisonnés par plateforme, offrent des capacités d'analyse limitées et ne permettent ni la consolidation automatique des indicateurs, ni la génération de livrables de reporting prêts à être diffusés (rapports PDF, présentations, emails de synthèse). En pratique, la production d'un rapport de performance mensuel nécessite encore, dans de nombreuses organisations, une intervention humaine répétitive : extraction manuelle des données, copie dans des tableurs, calcul des indicateurs, mise en forme de graphiques, rédaction de commentaires et diffusion par courrier électronique. Ce processus est non seulement chronophage, mais également sujet à l'erreur humaine, peu réactif face aux anomalies critiques, et difficilement industrialisable à mesure que le portefeuille d'applications suivies s'accroît.

C'est dans ce contexte que s'inscrit le présent projet de fin d'études, dont l'objectif est de concevoir et de développer une plateforme intelligente d'automatisation du reporting analytique des applications mobiles. Cette plateforme repose sur trois piliers technologiques complémentaires : le Data Engineering, pour la collecte, le nettoyage, la transformation et le stockage structuré des données ; le Cloud Computing, pour l'hébergement, l'orchestration et la scalabilité de la solution ; et l'Intelligence Artificielle Générative, à travers l'intégration d'un grand modèle de langage (Large Language Model, LLM), pour produire des analyses

contextualisées, détecter des anomalies et formuler des recommandations exploitables par les équipes métier.

La problématique centrale de ce mémoire peut être formulée ainsi : comment concevoir une architecture logicielle et data capable d'automatiser de bout en bout la chaîne de valeur du reporting analytique mobile — de la collecte brute des données jusqu'à la diffusion de livrables intelligents — tout en garantissant la fiabilité, la scalabilité et la pertinence analytique du système ? Cette problématique se décline en plusieurs questions de recherche appliquée : Quelles architectures de collecte de données permettent d'interagir de manière fiable avec les API hétérogènes de Google Play et de l'App Store ? Quel modèle de données relationnel est le plus adapté pour supporter à la fois le stockage historique et les besoins de Business Intelligence ? Dans quelle mesure un grand modèle de langage peut-il enrichir un pipeline analytique traditionnel en produisant des interprétations pertinentes et fiables ? Quelle orchestration technique permet d'automatiser l'ensemble du pipeline sans sacrifier la maintenabilité du système ?

Pour répondre à ces questions, ce mémoire est structuré en cinq chapitres. Le premier chapitre dresse un état de l'art des domaines mobilisés par le projet : applications mobiles et App Store Optimization, Business Intelligence, Data Engineering, Cloud Computing, Big Data, automatisation des workflows et grands modèles de langage. Le deuxième chapitre présente l'analyse des besoins, à travers l'étude de l'existant, l'identification des acteurs et des cas d'utilisation, ainsi que la spécification des besoins fonctionnels et non fonctionnels. Le troisième chapitre expose la conception détaillée de la solution : architecture globale, architecture Cloud et data, modélisation UML, conception du schéma de données en étoile. Le quatrième chapitre décrit l'implémentation technique de la plateforme, technologie par technologie, en justifiant chaque choix par rapport à des alternatives concurrentes. Le cinquième chapitre présente les résultats obtenus, les livrables générés par la plateforme, ainsi qu'une analyse critique des performances du système. Le mémoire se conclut par une synthèse des contributions, des limites identifiées et des perspectives d'évolution.

Chapitre 1 — État de l'art

Introduction

Ce premier chapitre a pour objectif de poser les fondations théoriques et technologiques nécessaires à la compréhension du projet. Il présente successivement l'écosystème des applications mobiles et les pratiques d'App Store Optimization, les concepts fondamentaux de la Business Intelligence et du Data Engineering, les principes du Cloud Computing et du Big Data, les architectures d'entrepôt de données et les paradigmes ETL/ELT, l'automatisation des workflows métier, puis les grands modèles de langage et le Prompt Engineering. Une synthèse comparative des solutions existantes clôt ce chapitre et permet de positionner la contribution du présent projet.

1.1 Les applications mobiles et leurs écosystèmes de distribution

Une application mobile est un logiciel conçu pour s'exécuter sur un terminal mobile (smartphone ou tablette), distribué principalement via des plateformes centralisées appelées magasins d'applications (app stores). Les deux écosystèmes dominants sont Android, porté par Google, et iOS, porté par Apple. Chacun de ces écosystèmes impose ses propres règles de publication, ses formats de distribution (APK/AAB pour Android, IPA pour iOS) et ses outils d'administration dédiés.

1.1.1 Google Play Console

Google Play Console est la plateforme d'administration mise à disposition des développeurs Android pour publier, suivre et analyser leurs applications sur Google Play. Elle propose nativement des rapports statistiques concernant les installations, les désinstallations, les avis, les notes, les revenus, la stabilité (taux de plantage, ANR — Application Not Responding) ainsi que des données d'acquisition. Google Play propose également une API de reporting permettant l'extraction programmatique de certains de ces indicateurs, généralement sous forme de fichiers exportés vers un espace de stockage Cloud (Google Cloud Storage).

1.1.2 App Store Connect

App Store Connect est l'équivalent proposé par Apple pour la gestion des applications iOS. Il fournit des tableaux de bord d'analyse (App Analytics) portant sur les impressions, les pages de fiche produit consultées, les téléchargements, les sessions actives, ainsi que les ventes et

les avis. Apple met à disposition une API REST authentifiée par jeton JWT (App Store Connect API) permettant l'automatisation de la récupération de ces données, bien que la granularité et la latence de mise à disposition diffèrent sensiblement de celles de Google Play.

Tableau 1. Comparaison entre Google Play Console et App Store Connect

Critère	Google Play Console	App Store Connect
Authentification API	Compte de service Google (OAuth2 / clé JSON)	Clé privée + JWT signé
Format d'export	Rapports CSV/JSON via Cloud Storage	Rapports via API REST (JSON)
Latence des données	24 à 48 heures selon le rapport	24 à 48 heures selon le rapport
Granularité géographique	Pays / région	Pays / région / territoire
Indicateurs de plantage	Oui (Android Vitals)	Oui (Xcode Organizer / App Analytics)
Gestion des avis	API dédiée (Reviews API)	API dédiée (Customer Reviews)

1.2 App Store Optimization (ASO)

L'App Store Optimization désigne l'ensemble des pratiques visant à améliorer la visibilité et le taux de conversion d'une application au sein des résultats de recherche et des pages de recommandation des magasins d'applications. L'ASO repose sur l'optimisation de métadonnées (titre, description, mots-clés, visuels), mais également sur le suivi rigoureux d'indicateurs de performance tels que le taux de conversion fiche-vers-installation, le taux de rétention à J1/J7/J30, ou encore la note moyenne pondérée par le volume d'avis. La littérature spécialisée souligne que l'efficacité d'une démarche ASO dépend fortement de la capacité d'une organisation à disposer de données fiables, consolidées et mises à jour régulièrement — ce qui constitue précisément l'un des enjeux adressés par la plateforme développée dans ce projet.



Figure 1. Cycle de vie de l'optimisation ASO (mots-clés, conversion, rétention, itération)

1.3 Business Intelligence (BI)

La Business Intelligence regroupe l'ensemble des méthodes, technologies et outils permettant de collecter, structurer et analyser les données d'une organisation afin d'appuyer la prise de décision. Une chaîne de valeur BI classique comprend généralement quatre étapes : la collecte des données sources (souvent hétérogènes), leur intégration au sein d'un entrepôt de données, le calcul d'indicateurs clés de performance (KPIs), et enfin leur restitution via des tableaux de bord ou des rapports destinés aux utilisateurs métier.

Dans le contexte du reporting analytique mobile, la BI permet de transformer des données brutes d'usage (installations, sessions, avis) en informations actionnables (taux de croissance, taux de churn, score de satisfaction) directement exploitables par les équipes produit et marketing. La plateforme développée dans ce mémoire s'inscrit pleinement dans cette logique, en automatisant l'intégralité de la chaîne BI, de la collecte à la restitution.

1.4 Data Engineering

Le Data Engineering est la discipline de l'ingénierie logicielle consacrée à la conception, la construction et la maintenance des infrastructures et pipelines permettant de collecter, transporter, transformer et stocker des données à grande échelle, dans des conditions de fiabilité, de performance et de qualité maîtrisées. Un Data Engineer conçoit des pipelines de données (data pipelines) qui automatisent le passage d'une donnée depuis sa source (API, base de données, fichier) jusqu'à sa destination finale (entrepôt de données, tableau de bord, modèle d'IA).

Les principales responsabilités du Data Engineering incluent : l'extraction de données depuis des sources hétérogènes (API REST, bases de données, fichiers plats), le nettoyage et la validation des données (data cleaning, data quality), la transformation des données (normalisation, agrégation, enrichissement), l'orchestration de pipelines automatisés, et la conception de modèles de stockage adaptés aux usages analytiques (schémas en étoile, en flocon, data vault). Ce projet mobilise l'ensemble de ces compétences, appliquées spécifiquement au domaine des données d'applications mobiles.

1.5 Cloud Computing

Le Cloud Computing désigne la mise à disposition, via Internet, de ressources informatiques (calcul, stockage, réseau, services applicatifs) à la demande, généralement facturées à l'usage. Le National Institute of Standards and Technology (NIST) définit le Cloud Computing selon cinq caractéristiques essentielles : le libre-service à la demande, l'accès réseau étendu, la mutualisation des ressources, l'élasticité rapide et la mesure du service. Trois modèles de service sont généralement distingués :

Infrastructure as a Service (IaaS) : mise à disposition de ressources de calcul, de stockage et de réseau virtualisées (ex. machines virtuelles) ;

Platform as a Service (PaaS) : mise à disposition d'un environnement d'exécution géré, incluant système d'exploitation, middleware et runtime applicatif ;

Software as a Service (SaaS) : mise à disposition d'une application complète, accessible via un navigateur ou une API, sans gestion de l'infrastructure sous-jacente par le client.

La plateforme développée dans ce projet s'appuie sur des services de type PaaS et SaaS (Supabase pour la base de données managée, n8n pour l'orchestration, OpenAI pour l'inférence du modèle de langage), ce qui permet de réduire considérablement la charge

opérationnelle liée à la gestion d'infrastructure et de se concentrer sur la logique métier du pipeline.

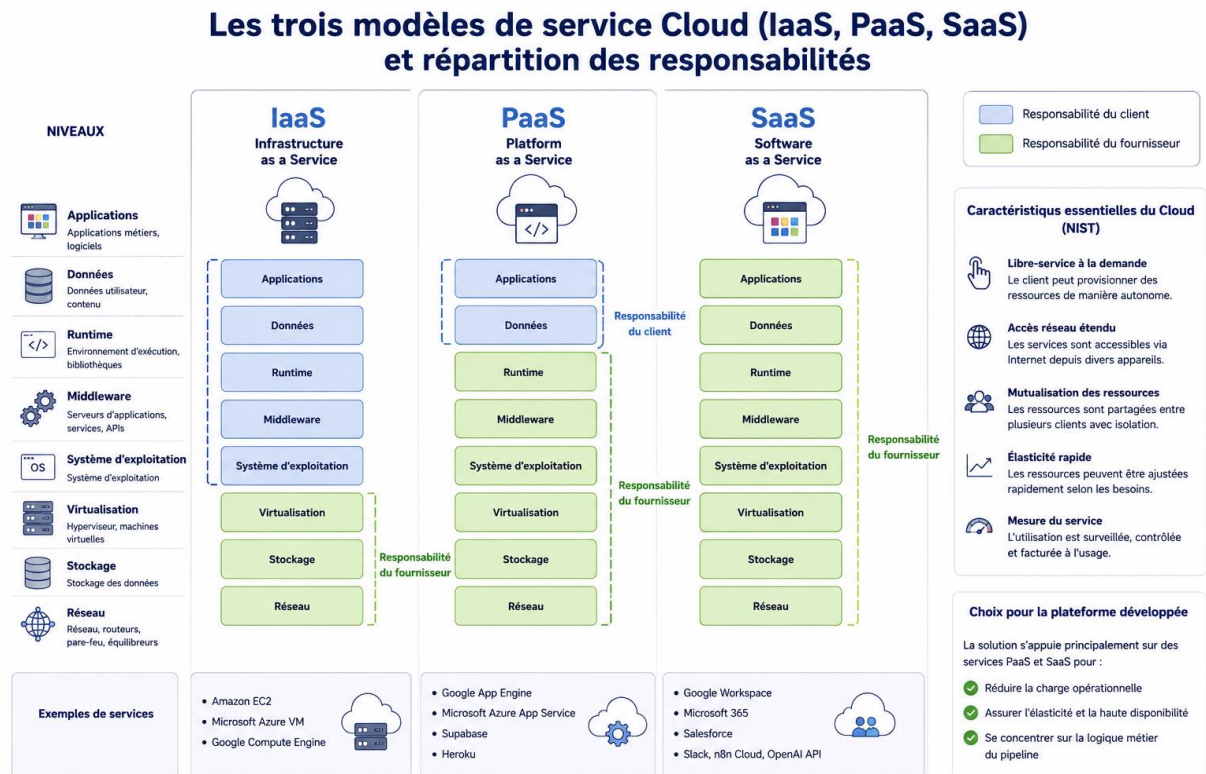


Figure 2. Les trois modèles de service Cloud (IaaS, PaaS, SaaS) et répartition des responsabilités

1.6 Big Data

Le Big Data désigne des ensembles de données dont le volume, la vitesse de génération ou la variété dépassent les capacités de traitement des outils traditionnels de gestion de données. Ces caractéristiques sont classiquement résumées par le modèle des « 5V » : Volume, Vitesse (Velocity), Variété, Véracité et Valeur. Si les volumes de données traités dans le cadre de ce projet ne relèvent pas nécessairement du Big Data au sens strict (téraoctets, flux temps réel massif), les principes architecturaux issus de cet écosystème — pipelines découplés, traitement par lots (batch processing), scalabilité horizontale — demeurent pleinement pertinents et ont guidé la conception de la solution, dans une logique de préparation à la montée en charge.

1.7 Entrepôts de données et modélisation dimensionnelle

Un entrepôt de données (Data Warehouse) est une base de données spécifiquement conçue pour les besoins d'analyse décisionnelle, par opposition aux bases de données

transactionnelles (OLTP) optimisées pour les opérations courantes. Les entrepôts de données reposent généralement sur une modélisation dimensionnelle, dont les deux schémas les plus répandus sont le schéma en étoile (Star Schema) et le schéma en flocon (Snowflake Schema).

Le schéma en étoile organise les données autour d'une ou plusieurs tables de faits (contenant les mesures quantitatives, par exemple le nombre d'installations journalières) reliées à des tables de dimensions (contenant les attributs descriptifs, par exemple l'application, la plateforme, la date ou le pays). Ce modèle, retenu pour la conception de la base de données de ce projet et détaillé au chapitre 3, offre un excellent compromis entre simplicité de compréhension, performance des requêtes analytiques et facilité de maintenance.

Tableau 2. Comparaison entre schéma en étoile et schéma en flocon

Critère	Schéma en étoile	Schéma en flocon
Structure des dimensions	Dénormalisée (une seule table par dimension)	Normalisée (dimensions décomposées en sous-tables)
Complexité des requêtes	Faible (moins de jointures)	Plus élevée (jointures multiples)
Performance en lecture	Généralement supérieure	Peut être inférieure sur de gros volumes
Redondance des données	Plus élevée	Réduite
Facilité de compréhension métier	Élevée	Moyenne

1.8 Paradigmes ETL et ELT

ETL (Extract, Transform, Load) et ELT (Extract, Load, Transform) sont deux paradigmes d'intégration de données qui diffèrent par l'ordre des opérations de transformation et de chargement. Dans une architecture ETL traditionnelle, les données sont transformées avant d'être chargées dans l'entrepôt cible, généralement à l'aide d'un serveur de transformation dédié. Dans une architecture ELT, les données brutes sont d'abord chargées telles quelles dans le système cible, puis transformées in situ, en tirant parti de la puissance de calcul de ce dernier (par exemple via des vues ou des requêtes SQL).

L'essor des entrepôts de données Cloud modernes (Snowflake, BigQuery, ou bases PostgreSQL managées comme Supabase) a favorisé l'adoption du paradigme ELT, qui offre davantage de flexibilité et de traçabilité, en conservant les données brutes disponibles pour d'éventuels retraitements ultérieurs. La plateforme développée dans ce projet adopte une approche hybride : les données sont nettoyées et normalisées dès leur extraction (étape indispensable compte tenu de l'hétérogénéité des formats de sortie de Google Play et de l'App Store), puis chargées dans la base relationnelle, où des transformations analytiques supplémentaires (calcul de KPIs agrégés) sont effectuées.

Tableau 3. Comparaison entre les approches ETL et ELT

Critère	ETL	ELT
Lieu de transformation	Serveur de transformation dédié	Système de destination (entrepôt)
Conservation des données brutes	Non systématique	Oui, par défaut
Flexibilité analytique a posteriori	Limitée	Élevée
Adapté aux volumes massifs	Moyennement	Fortement (traitement distribué)
Complexité d'implémentation initiale	Plus élevée	Plus faible avec des outils Cloud modernes

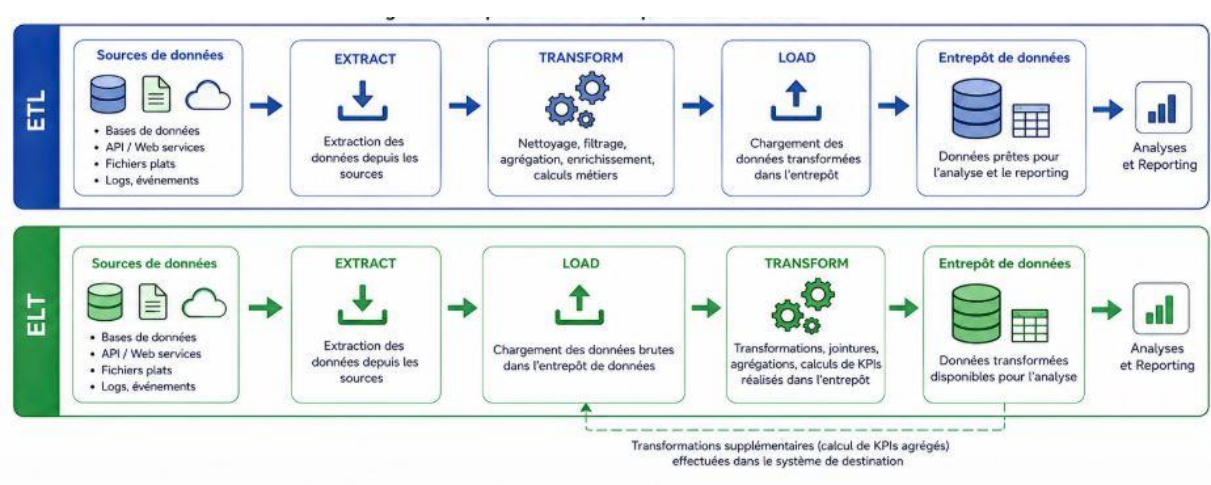


Figure 3. Comparaison schématique des flux ETL et ELT

1.9 Automatisation des workflows (Workflow Automation)

L'automatisation des workflows consiste à orchestrer, sans intervention humaine répétitive, un enchaînement d'étapes techniques (appels API, transformations, écritures en base, notifications) déclenché par un événement ou une planification temporelle. Les plateformes d'automatisation modernes, dites low-code/no-code, telles que n8n, Zapier ou Make (anciennement Integromat), permettent de construire visuellement des workflows sous forme de graphes de nœuds interconnectés, chaque nœud représentant une action ou une transformation. Ces outils réduisent considérablement le temps de développement par rapport à une orchestration codée manuellement, tout en conservant, pour certains d'entre eux comme n8n, la possibilité d'insérer du code personnalisé (JavaScript ou Python) pour les traitements spécifiques.

1.10 Grands modèles de langage (LLM) et intelligence artificielle générative

Les grands modèles de langage (Large Language Models, LLM) sont des modèles d'apprentissage profond, généralement fondés sur l'architecture Transformer, entraînés sur de vastes corpus textuels afin de modéliser statistiquement le langage naturel. Ces modèles, dont les représentants les plus connus incluent la famille GPT (OpenAI), Claude (Anthropic) ou Gemini (Google), sont capables de générer du texte cohérent, de résumer des documents, de raisonner sur des données structurées fournies en contexte, et de produire des analyses en langage naturel à partir de données chiffrées.

Dans le cadre de ce projet, un LLM est intégré au pipeline analytique afin de transformer les indicateurs chiffrés (KPIs) en commentaires interprétatifs contextualisés, similaires à ceux qu'un analyste humain produirait lors de la rédaction d'un rapport de performance. Cette capacité constitue une contribution différenciante par rapport aux outils de reporting traditionnels, purement descriptifs.

1.10.1 Prompt Engineering

Le Prompt Engineering désigne l'ensemble des techniques permettant de formuler les instructions (prompts) transmises à un LLM de manière à maximiser la pertinence, la fiabilité et la structure de ses réponses. Parmi les techniques courantes figurent : le prompting par instructions explicites, le prompting par exemples (few-shot prompting), la décomposition en

étapes de raisonnement (chain-of-thought), ainsi que la contrainte de format de sortie (par exemple, exiger une réponse strictement structurée en JSON). Dans ce projet, le Prompt Engineering est utilisé pour garantir que les commentaires générés par le LLM respectent un format standardisé, s'appuient exclusivement sur les données fournies en contexte, et évitent les approximations ou les généralisations non fondées (phénomène d'hallucination).

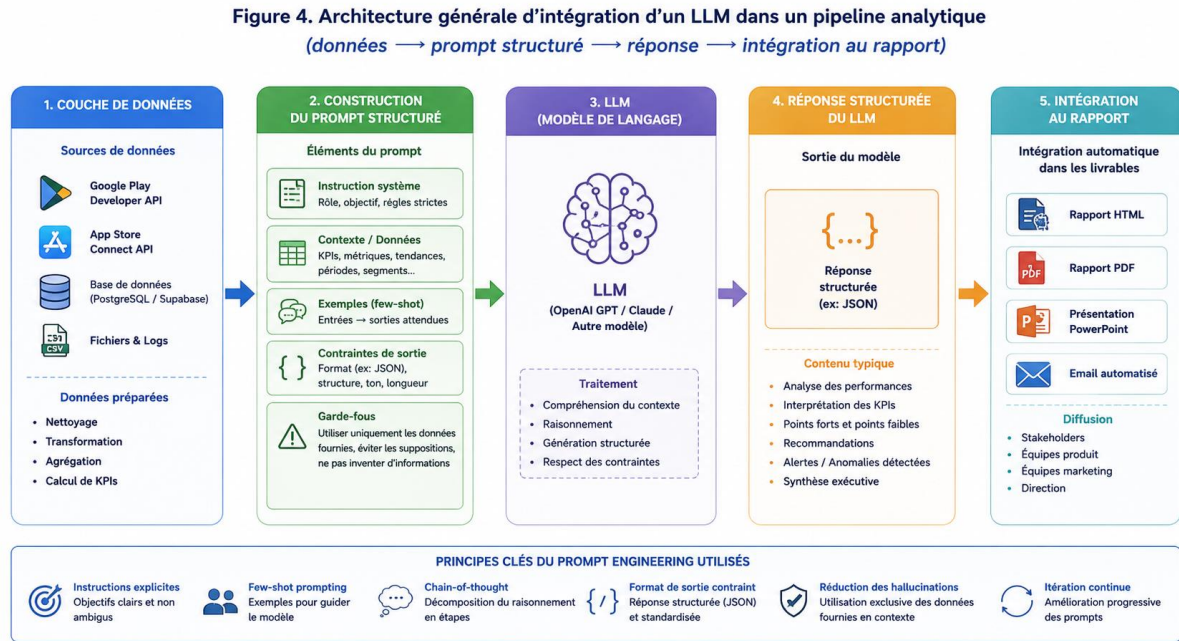


Figure 4. Architecture générale d'intégration d'un LLM dans un pipeline analytique (données → prompt structuré → réponse → intégration au rapport)

1.11 Synthèse comparative des solutions existantes

Plusieurs catégories d'outils existent aujourd'hui sur le marché pour répondre, partiellement, aux besoins de reporting analytique mobile : les consoles natives des plateformes (Google Play Console, App Store Connect), les outils spécialisés d'analytics mobile (Firebase, AppsFlyer, Adjust), les outils de Business Intelligence généralistes (Power BI, Tableau, Looker) et les plateformes d'automatisation no-code (Zapier, Make, n8n). Le tableau suivant synthétise leurs forces et limites respectives au regard des besoins identifiés pour ce projet.

Tableau 4. Comparaison des solutions existantes de reporting analytique mobile

Solution	Points forts	Limites principales
Consoles natives (Play Console / App Store)	Données officielles, gratuites, granularité fine	Pas de consolidation multi-plateforme, pas de génération

Connect)		automatique de livrables
Outils d'analytics mobile (Firebase, AppsFlyer)	Suivi comportemental avancé, attribution marketing	Coût élevé, ne couvrent pas la génération de rapports formatés (PDF/PPT)
Outils BI généralistes (Power BI, Tableau)	Visualisation puissante, connecteurs multiples	Nécessitent une intégration manuelle des sources, pas de couche IA générative native
Plateformes no-code (Zapier, Make, n8n)	Automatisation flexible, faible coût d'entrée	Nécessitent une conception sur-mesure du pipeline analytique et de la couche IA
Plateforme proposée (ce projet)	Pipeline ELT bout-en-bout, KPIs automatisés, couche IA générative, livrables multi-formats	Solution académique, périmètre initial limité à deux plateformes sources

Cette analyse comparative met en évidence un vide fonctionnel que la plateforme proposée dans ce mémoire vise à combler : l'absence, sur le marché, d'une solution intégrée combinant collecte automatisée multi-plateforme, calcul de KPIs, stockage structuré, génération de livrables multi-formats et couche d'intelligence artificielle générative pour l'interprétation des résultats.

Conclusion du chapitre

Ce chapitre a permis de poser les bases théoriques mobilisées par le projet, en couvrant l'écosystème des applications mobiles, les principes de la Business Intelligence et du Data Engineering, les fondamentaux du Cloud Computing et du Big Data, les paradigmes d'intégration de données (ETL/ELT), l'automatisation des workflows, ainsi que les grands modèles de langage et le Prompt Engineering. La synthèse comparative des solutions existantes a permis de mettre en évidence les limites des outils actuels et de justifier la pertinence du projet. Le chapitre suivant s'attache à traduire ce cadre théorique en une analyse concrète des besoins fonctionnels et non fonctionnels de la plateforme à concevoir.

Références bibliographiques du chapitre

- [1] Kimball, R., Ross, M. — The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, Wiley.
- [2] Mell, P., Grance, T. — The NIST Definition of Cloud Computing, National Institute of Standards and Technology.
- [3] Vaswani, A. et al. — Attention Is All You Need, Advances in Neural Information Processing Systems.
- [4] Reis, J., Housley, M. — Fundamentals of Data Engineering, O'Reilly Media.
- [5] Documentation officielle Google Play Console — support.google.com/googleplay/android-developer
- [6] Documentation officielle App Store Connect API — developer.apple.com/documentation/appstoreconnectapi

Chapitre 2 — Analyse des besoins

Introduction

Ce chapitre a pour objectif de formaliser les besoins auxquels la plateforme doit répondre. Il s'ouvre sur une présentation du problème et une étude critique du processus manuel actuellement observé dans les organisations gérant des applications mobiles, avant d'identifier les acteurs du système, les cas d'utilisation, ainsi que les besoins fonctionnels et non fonctionnels. Une modélisation BPMN du processus cible complète cette analyse.

2.1 Présentation du problème

Le suivi de la performance d'une application mobile mobilise, dans la majorité des organisations, un processus manuel répétitif consistant à se connecter successivement aux consoles Google Play Console et App Store Connect, à exporter des fichiers de données (souvent au format CSV), à les consolider dans un tableur, à calculer manuellement des indicateurs (croissance des installations, évolution de la note moyenne, taux de désinstallation), puis à mettre en forme ces résultats sous forme de rapport ou de présentation, avant de les diffuser par courrier électronique aux parties prenantes. Ce processus présente plusieurs limites majeures détaillées ci-après.

2.2 Étude de l'existant et processus manuel

Le processus manuel actuellement observé peut être décomposé en six étapes successives : (1) connexion aux consoles des plateformes et export des données brutes ; (2) téléchargement et consolidation des fichiers exportés dans un tableur ; (3) nettoyage manuel des données (suppression de doublons, harmonisation des formats de date, gestion des valeurs manquantes) ; (4) calcul manuel des indicateurs de performance à l'aide de formules ; (5) mise en forme de graphiques et rédaction de commentaires d'analyse ; (6) diffusion du rapport final par courrier électronique ou lors d'une réunion. Ce processus est généralement répété à une fréquence hebdomadaire ou mensuelle, pour chaque application suivie.

Figure 1. Représentation du processus manuel actuel de reporting analytique mobile

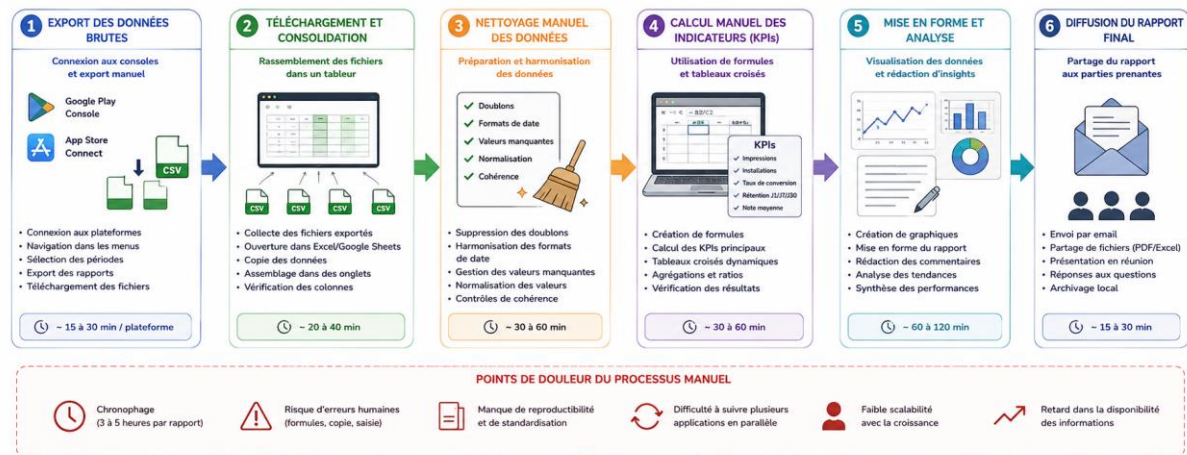


Figure 1. Représentation du processus manuel actuel de reporting analytique mobile

2.3 Analyse critique de l'existant

L'analyse du processus manuel met en évidence plusieurs faiblesses structurelles :

Chronophagie : la production d'un rapport complet peut nécessiter plusieurs heures de travail humain par application suivie ;

Risque d'erreur : la manipulation manuelle de données dans un tableur expose à des erreurs de saisie, de formule ou de copier-coller ;

Manque de réactivité : les anomalies critiques (chute brutale de la note moyenne, pic de désinstallations) ne sont détectées qu'au moment de la production du rapport périodique, avec un délai de réaction potentiellement élevé ;

Absence de consolidation multi-plateforme : les données Android et iOS sont rarement analysées de manière unifiée, faute d'un modèle de données commun ;

Non-scalabilité : le processus manuel devient rapidement intenable lorsque le nombre d'applications suivies augmente ;

Absence d'interprétation contextualisée : les rapports produits se limitent souvent à une restitution chiffrée, sans analyse qualitative structurée des causes probables des variations observées.

2.4 Contraintes du projet

Le développement de la plateforme est soumis à plusieurs contraintes : contrainte de temps, le projet devant être réalisé dans le cadre d'un semestre de Projet de Fin d'Études ; contrainte

technologique, les API de Google Play et de l'App Store imposant des formats et des modes d'authentification spécifiques et parfois peu documentés ; contrainte budgétaire, la solution devant privilégier des services Cloud proposant des offres gratuites ou à faible coût, compatibles avec un contexte académique ; contrainte de confidentialité, aucune donnée réelle d'entreprise commerciale ne pouvant être utilisée dans le cadre de ce mémoire académique, la validation étant réalisée sur des données simulées représentatives.

2.5 Acteurs du système

Tableau 1. Identification des acteurs du système

Acteur	Description	Interaction principale
Responsable produit / marketing	Utilisateur final destinataire des rapports	Reçoit les rapports PDF, présentations et emails générés
Administrateur de la plateforme	Configure les applications suivies et les paramètres du pipeline	Interagit avec l'interface d'administration et n8n
Système Google Play	Source de données externe	Fournit les données via API/export
Système App Store Connect	Source de données externe	Fournit les données via API REST
Moteur d'orchestration (n8n)	Automatise l'exécution du pipeline	Déclenche les tâches selon planification ou événement
Modèle de langage (LLM)	Génère les commentaires analytiques	Reçoit un prompt structuré et retourne une analyse
Base de données (Supabase/PostgreSQL)	Stocke les données historisées	Alimentée par le pipeline, interrogée pour le reporting

2.6 Diagramme de contexte

Le diagramme de contexte suivant positionne la plateforme au centre de ses interactions avec les systèmes externes (Google Play, App Store Connect, service de messagerie) et les utilisateurs finaux.

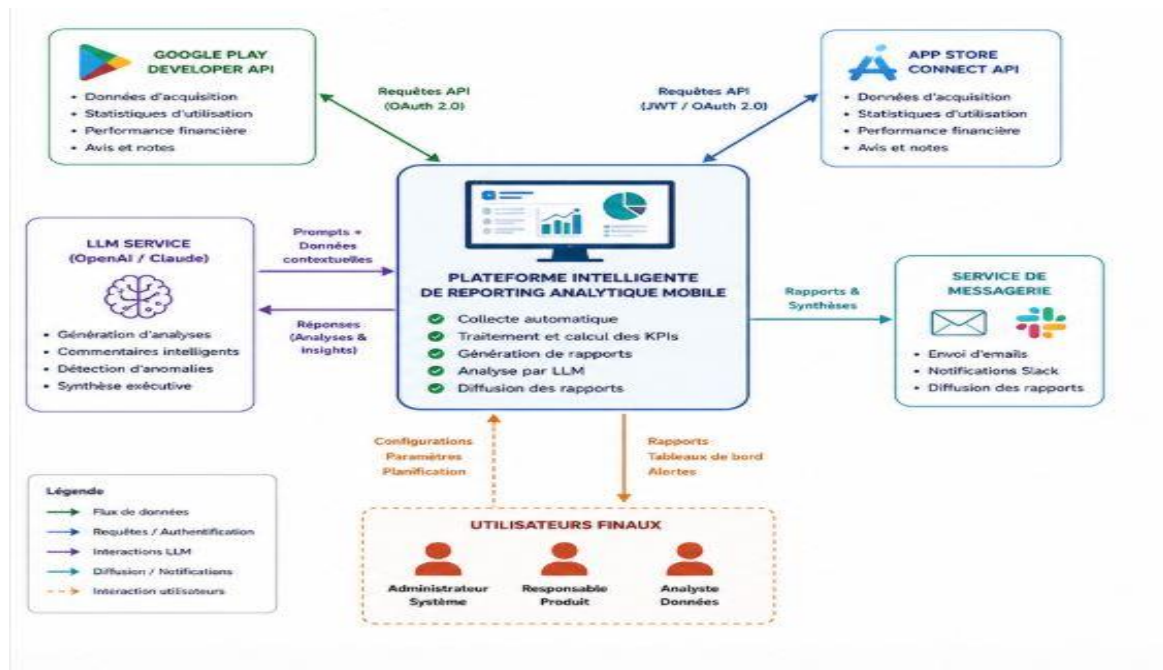


Figure 2. Diagramme de contexte du système

2.7 Cas d'utilisation (diagramme UML)

Le diagramme de cas d'utilisation ci-dessous synthétise les principales fonctionnalités offertes par la plateforme du point de vue des différents acteurs identifiés.

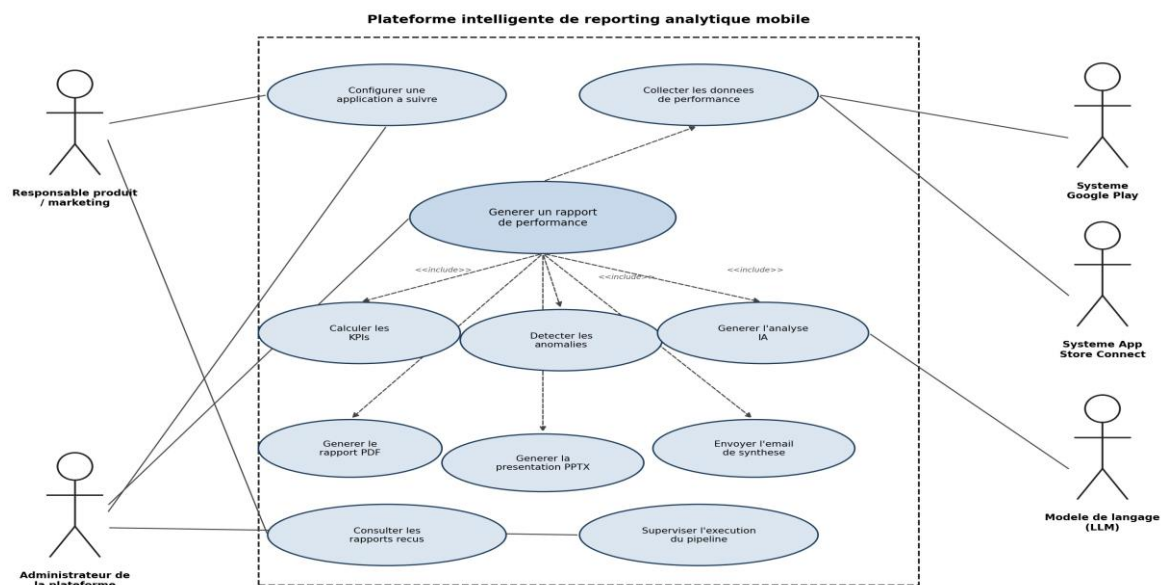


Figure 3. Diagramme de cas d'utilisation UML de la plateforme

Les principaux cas d'utilisation identifiés sont détaillés ci-après :

Tableau 2. Description du cas d'utilisation « Générer un rapport de performance »

Champ	Description
Acteur principal	Système (déclenchement automatique planifié) ou Administrateur (déclenchement manuel)
Pré-condition	Les données de la période concernée ont été collectées et validées
Scénario nominal	1. Le système calcule les KPIs de la période. 2. Le système interroge le LLM pour produire une analyse. 3. Le système génère le rapport PDF et la présentation PowerPoint. 4. Le système envoie l'email de synthèse.
Scénario alternatif	Si les données sont incomplètes, le système signale l'anomalie et suspend la génération jusqu'à résolution
Post-condition	Le rapport, la présentation et l'email sont générés et archivés

2.8 Besoins fonctionnels

Les besoins fonctionnels identifiés sont numérotés de BF-01 à BF-10 afin de faciliter leur traçabilité tout au long du mémoire.

Réf.	Besoin fonctionnel
BF-01	Collecter automatiquement les données depuis l'API Google Play (installations, désinstallations, notes, avis, revenus, plantages).
BF-02	Collecter automatiquement les données depuis l'API App Store Connect (téléchargements, sessions, ventes, avis).
BF-03	Nettoyer et normaliser les données collectées (dédoublonnage, harmonisation des formats, gestion des valeurs manquantes).
BF-04	Calculer automatiquement les indicateurs clés de performance (KPIs) à fréquence journalière, hebdomadaire et mensuelle.
BF-05	Stocker les données brutes et transformées dans une base de données relationnelle structurée.
BF-06	Détecter automatiquement les anomalies significatives (variations dépassant un seuil

	paramétrable).
BF-07	Générer une analyse textuelle contextualisée à l'aide d'un modèle de langage, à partir des KPIs calculés.
BF-08	Générer automatiquement un rapport PDF structuré et lisible.
BF-09	Générer automatiquement une présentation PowerPoint synthétique.
BF-10	Générer et envoyer automatiquement un email de synthèse aux destinataires configurés.

2.9 Besoins non fonctionnels

Réf.	Catégorie	Description
BNF-01	Fiabilité	Le pipeline doit garantir l'intégrité des données à chaque étape (validation, journalisation des erreurs).
BNF-02	Performance	Le traitement complet d'une exécution de pipeline ne doit pas excéder quelques minutes pour un portefeuille d'applications de taille modérée.
BNF-03	Scalabilité	L'architecture doit permettre l'ajout de nouvelles applications ou plateformes sans refonte majeure.
BNF-04	Sécurité	Les identifiants d'accès aux API tierces doivent être stockés de manière chiffrée et jamais exposés en clair.
BNF-05	Maintenabilité	Le pipeline doit être modulaire, documenté et facilement modifiable par un futur développeur.
BNF-06	Disponibilité	Le système doit pouvoir être exécuté selon une planification régulière sans intervention manuelle.
BNF-07	Portabilité	La solution doit reposer sur des technologies Cloud portables, évitant une dépendance excessive à un fournisseur unique.

2.10 User Stories

Les besoins ont également été formalisés sous forme de user stories, selon le format « En tant que... je veux... afin de... », usuel dans les démarches agiles.

En tant que responsable marketing, je veux recevoir chaque semaine un email de synthèse afin de suivre la performance de mon application sans consulter manuellement les consoles.

En tant qu'administrateur de la plateforme, je veux pouvoir ajouter une nouvelle application à suivre afin d'étendre le périmètre du reporting sans développement supplémentaire.

En tant que responsable produit, je veux être alerté automatiquement en cas d'anomalie critique afin de réagir rapidement à un problème de qualité.

En tant que direction, je veux disposer d'une présentation PowerPoint synthétique afin de partager les résultats lors des comités de pilotage.

2.11 Modélisation BPMN du processus cible

Le diagramme BPMN (Business Process Model and Notation) ci-dessous modélise le processus automatisé cible, en distinguant les couloirs (swimlanes) correspondant respectivement au déclencheur planifié, au pipeline de collecte et de transformation, au module d'intelligence artificielle, et à la diffusion des livrables.

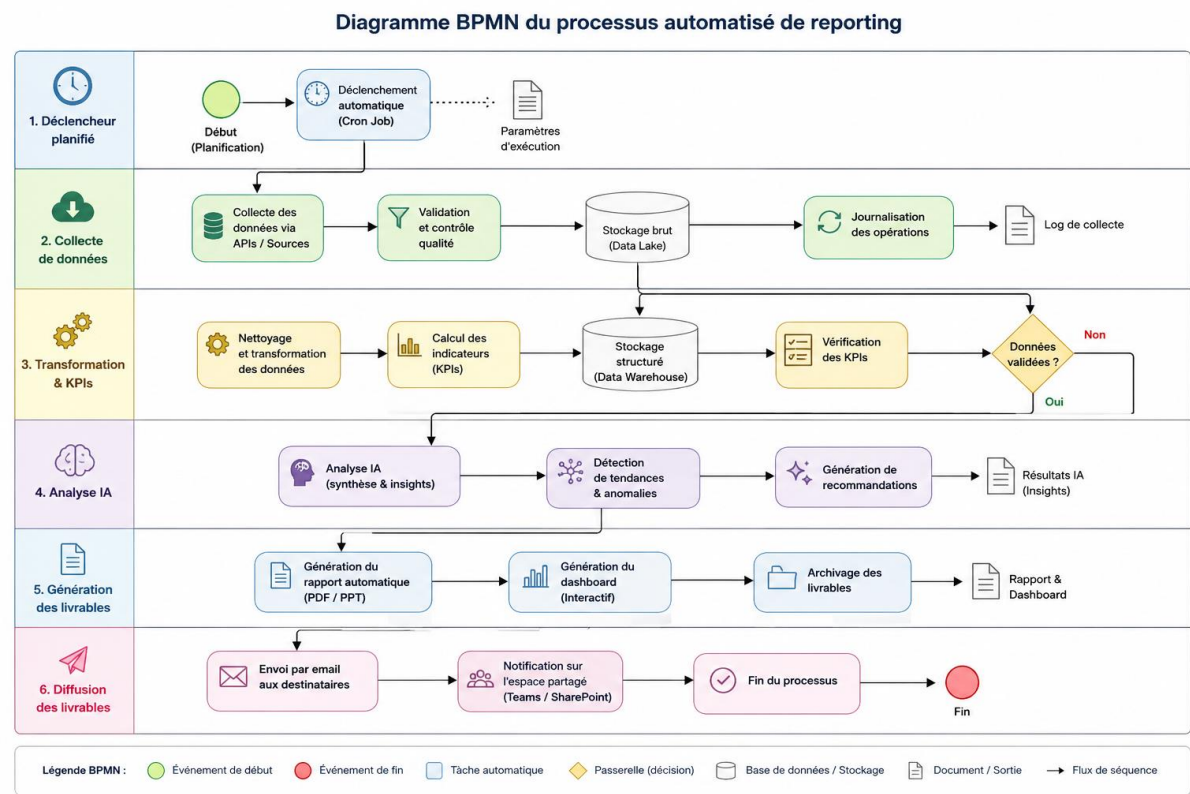


Figure 4. Diagramme BPMN du processus automatisé de reporting

Conclusion du chapitre

Ce chapitre a permis de formaliser précisément les besoins fonctionnels et non fonctionnels de la plateforme, à partir d'une analyse critique du processus manuel actuellement observé. L'identification des acteurs, la modélisation des cas d'utilisation et du processus cible en BPMN constituent la base de la phase de conception, qui fait l'objet du chapitre suivant.

Références bibliographiques du chapitre

- [1] OMG — Business Process Model and Notation (BPMN) Specification, Object Management Group.
- [2] Cockburn, A. — Writing Effective Use Cases, Addison-Wesley.
- [3] Cohn, M. — User Stories Applied: For Agile Software Development, Addison-Wesley.

Chapitre 3 — Étude comparative des technologies

Introduction

Alors que le chapitre précédent a permis de spécifier les besoins fonctionnels et non fonctionnels de la plateforme proposée, ce chapitre a pour vocation d'objectiver les choix technologiques retenus pour la réalisation du cadre expérimental. Pour chaque brique technologique du système, une présentation générale est proposée, suivie d'une comparaison argumentée avec au moins deux alternatives concurrentes, et d'une justification du choix retenu au regard des contraintes et besoins identifiés au chapitre 2. Cette démarche comparative, inspirée des méthodes de sélection multicritère utilisées en ingénierie logicielle, vise à conférer au processus de choix technologique une rigueur scientifique plutôt qu'une simple préférence subjective.

3.1 Langage de programmation : Python

Python est un langage interprété, multi-paradigme, dont l'écosystème de bibliothèques dédiées au traitement de données (pandas, NumPy), à l'automatisation (requests, schedule) et à l'intelligence artificielle (openai, anthropic, langchain) en fait un candidat naturel pour l'implémentation de pipelines de Data Engineering. Sa syntaxe concise et sa vaste communauté réduisent le temps de développement, ce qui constitue un facteur déterminant dans le contexte contraint d'un Projet de Fin d'Études.

Tableau 1. Comparaison des langages envisagés pour l'implémentation du pipeline

Critère	Python	Node.js (JavaScript)	Java
Écosystème Data/IA	Très riche (pandas, openai, anthropic)	Moyen (SDKs disponibles mais moins matures pour la data)	Riche mais verbeux
Rapidité de développement	Élevée	Élevée	Plus faible
Performance brute	Moyenne	Bonne (I/O asynchrone)	Élevée
Intégration avec n8n (nœuds Code)	Supportée nativement	Supportée nativement	Non supportée nativement
Courbe d'apprentissage	Faible	Faible à moyenne	Élevée

Le choix de Python est justifié par la convergence de trois facteurs : la disponibilité de bibliothèques officielles pour l'intégration avec les API d'OpenAI et d'Anthropic, la compatibilité native avec les nœuds de code personnalisé de l'orchestrateur n8n retenu (cf. section 3.7), et la maîtrise préalable du langage dans le cadre du cursus académique suivi.

3.2 Système de gestion de base de données : PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle open source, reconnu pour sa conformité aux standards SQL, la richesse de son typage (JSONB, tableaux, types géométriques) et l'extensibilité de son moteur (extensions PostGIS, pg_cron, pgvector). Ces caractéristiques en font un candidat pertinent pour un système devant à la fois stocker des données relationnelles structurées (dimensions, faits) et, potentiellement, des données semi-structurées issues des réponses brutes des API tierces.

Tableau 2. Comparaison des systèmes de gestion de base de données envisagés

Critère	PostgreSQL	MySQL	MongoDB (NoSQL)
Modèle de données	Relationnel, support JSONB	Relationnel	Orienté documents
Adéquation avec le modèle en étoile	Élevée	Élevée	Faible (jointures limitées)
Écosystème Cloud managé	Très large (Supabase, RDS, Cloud SQL)	Large	Large (Atlas)
Row Level Security native	Oui	Non native	Non applicable
Conformité ACID	Complète	Complète (InnoDB)	Partielle selon configuration

Le choix de PostgreSQL découle directement de l'adéquation du modèle relationnel avec le schéma en étoile retenu pour la conception de l'entrepôt de données (cf. chapitre 5), ainsi que de la disponibilité d'un mécanisme natif de sécurité au niveau des lignes (Row Level Security), particulièrement pertinent pour l'isolement des données par application suivie.

3.3 Plateforme Backend-as-a-Service : Supabase

Supabase est une plateforme Backend-as-a-Service open source construite autour d'une instance PostgreSQL managée, à laquelle s'ajoutent des services complémentaires : authentification, stockage de fichiers, API REST et temps réel générées automatiquement à partir du schéma de base de données, ainsi qu'un mécanisme de sécurité au niveau des lignes (RLS).

Tableau 3. Comparaison des plateformes Backend-as-a-Service envisagées

Critère	Supabase	Firebase (Google)	PostgreSQL auto-hébergé
Modèle de données	Relationnel (PostgreSQL)	NoSQL (Firestore)	Relationnel
Génération automatique d'API REST	Oui (PostgREST)	Oui (SDK propriétaire)	Non (à développer)
Coût d'entrée	Offre gratuite généreuse	Offre gratuite limitée en lecture/écriture	Coût d'hébergement à la charge du porteur de projet
Portabilité (éviter le verrouillage propriétaire)	Élevée (SQL standard)	Faible (format propriétaire)	Élevée
Effort opérationnel (maintenance, sauvegardes)	Faible (managé)	Faible (managé)	Élevé

Le choix de Supabase est justifié par sa capacité à combiner la robustesse et la portabilité du modèle relationnel PostgreSQL avec la simplicité opérationnelle d'un service managé, sans imposer les contraintes de modélisation NoSQL propres à Firebase, peu compatibles avec l'approche dimensionnelle retenue pour l'entrepôt de données.

3.4 Conteneurisation : Docker

Docker est une technologie de conteneurisation permettant d'empaqueter une application ainsi que l'ensemble de ses dépendances dans une unité isolée et portable, appelée conteneur, exécutable de manière identique quel que soit l'environnement hôte. Dans le cadre du projet,

Docker est utilisé pour l'hébergement de l'instance n8n ainsi que pour la reproductibilité de l'environnement de développement.

Tableau 4. Comparaison des approches de déploiement envisagées

Critère	Docker (conteneurs)	Machine virtuelle classique	Déploiement natif (bare-metal)
Isolation des dépendances	Élevée	Élevée	Faible
Temps de démarrage	Quelques secondes	Quelques minutes	Immédiat mais dépendances partagées
Portabilité entre environnements	Élevée (image standardisée)	Moyenne	Faible
Consommation de ressources	Faible (partage du noyau hôte)	Élevée (OS complet par VM)	Faible

La conteneurisation via Docker a été retenue pour garantir la reproductibilité du cadre expérimental, condition essentielle à la validation scientifique des résultats présentés au chapitre 9, tout en limitant la consommation de ressources par rapport à une virtualisation complète.

3.5 Gestion de versions : Git et GitHub

Git est un système de gestion de versions décentralisé permettant de suivre l'historique des modifications apportées au code source, de travailler en parallèle sur des branches isolées et de fusionner les contributions de manière contrôlée. GitHub est une plateforme d'hébergement de dépôts Git, enrichie de fonctionnalités collaboratives (revue de code, intégration continue, gestion des versions publiées). L'utilisation de Git/GitHub pour ce projet répond à un impératif de traçabilité scientifique : chaque évolution du pipeline peut être documentée, historisée et, si nécessaire, restaurée à un état antérieur, ce qui constitue une bonne pratique reconnue en ingénierie logicielle et en recherche reproductible.

3.6 Interfaçage avec les API : REST, OAuth 2.0 et JWT

L'architecture proposée repose intégralement sur des échanges de données via des API REST (Representational State Transfer), un style d'architecture logicielle fondé sur des principes

d'uniformité d'interface, de statelessness (absence d'état conservé côté serveur entre deux requêtes) et de manipulation de ressources identifiées par des URI. L'authentification auprès des API tierces mobilisées (Google Play Developer Reporting API, App Store Connect API) repose sur deux mécanismes complémentaires : le protocole OAuth 2.0 pour Google, et un jeton signé JWT (JSON Web Token) pour Apple.

Le protocole OAuth 2.0, normalisé par la RFC 6749 de l'Internet Engineering Task Force, définit un cadre d'autorisation déléguée permettant à une application tierce d'accéder à des ressources protégées pour le compte d'un utilisateur, sans que ce dernier ne communique directement ses identifiants à l'application cliente. Dans le cas de Google Play, l'authentification s'effectue via un compte de service, mécanisme adapté aux communications machine-à-machine sans intervention humaine récurrente.

Le format JWT, normalisé par la RFC 7519, définit une structure de jeton compacte, auto-suffisante et signée numériquement, permettant de transmettre de manière sécurisée un ensemble d'informations (claims) entre deux parties. Dans le cas d'App Store Connect, un jeton JWT est généré côté client à partir d'une clé privée fournie par Apple, puis transmis dans l'en-tête d'autorisation de chaque requête API, avec une durée de validité limitée (20 minutes maximum), imposant un mécanisme de renouvellement automatique intégré au pipeline.

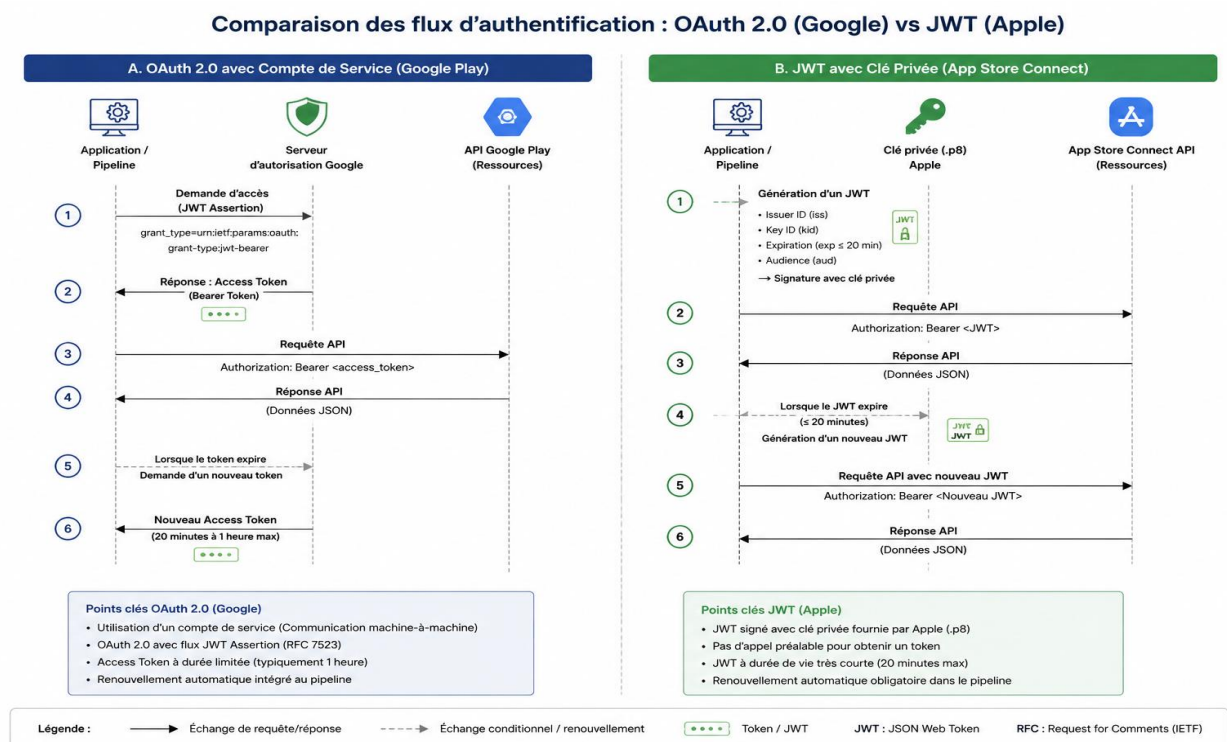


Figure 1. Comparaison schématique des flux d'authentification OAuth 2.0 (Google) et JWT (Apple)

Tableau 5. Comparaison des mécanismes d'authentification retenus

Critère	OAuth 2.0 (Google Play)	JWT signé (App Store Connect)
Nature du jeton	Jeton d'accès délivré par un serveur d'autorisation	Jeton auto-signé par le client à partir d'une clé privée
Durée de validité typique	1 heure (renouvelable via refresh token)	20 minutes maximum
Complexité d'implémentation	Moyenne (bibliothèque officielle Google disponible)	Moyenne (signature cryptographique ES256)
Renouvellement	Automatique via refresh token	Génération d'un nouveau jeton à chaque expiration

3.7 Orchestration de workflows : n8n

n8n est une plateforme d'automatisation de workflows open source, permettant de construire visuellement des pipelines sous forme de graphes de nœuds interconnectés, chaque nœud représentant une action (appel API, requête base de données, exécution de code personnalisé, envoi d'email). Contrairement à des solutions purement no-code, n8n autorise l'insertion de nœuds de code JavaScript ou Python, offrant une flexibilité déterminante pour l'implémentation de traitements analytiques spécifiques.

Tableau 6. Comparaison des plateformes d'orchestration de workflows envisagées

Critère	n8n	Zapier	Make (Integromat)	Apache Airflow
Modèle de licence	Open source (self-hosted possible)	Propriétaire SaaS	Propriétaire SaaS	Open source
Insertion de code personnalisé	Oui (JavaScript/Python)	Limitée	Limitée	Oui (Python natif)
Coût pour un usage académique	Faible (auto-hébergement gratuit)	Élevé au-delà d'un usage limité	Élevé au-delà d'un usage limité	Faible mais complexité d'exploitation élevée

Courbe d'apprentissage	Faible à moyenne	Faible	Faible	Élevée
Adéquation à un pipeline data + IA	Élevée	Faible	Moyenne	Élevée mais surdimensionnée

Le choix de n8n repose sur un compromis optimal entre simplicité de construction visuelle du pipeline, possibilité d'y intégrer du code personnalisé pour les traitements analytiques avancés, coût nul dans une configuration auto-hébergée, et complexité d'exploitation nettement inférieure à celle d'un orchestrateur de niveau industriel tel qu'Apache Airflow, disproportionné au regard du périmètre du cadre expérimental.

3.8 Génération de documents : HTML, CSS et Puppeteer

La génération du rapport HTML repose sur la construction dynamique d'un document structuré (HTML) mis en forme via des feuilles de style (CSS), à partir d'un gabarit (template) alimenté par les données calculées par le pipeline. Ce document HTML sert de base à la génération du rapport PDF, via l'outil Puppeteer, une bibliothèque Node.js pilotant une instance headless du navigateur Chromium, capable de restituer fidèlement une page HTML/CSS avant de l'exporter au format PDF.

Tableau 7. Comparaison des approches de génération de rapports PDF

Critère	HTML/CSS + Puppeteer	Bibliothèque PDF programmative (ex. ReportLab)	Modèle Word/LibreOffice converti
Fidélité de mise en forme (CSS avancé)	Élevée	Limitée (positionnement manuel)	Moyenne
Facilité de maintenance du gabarit	Élevée (HTML/CSS standard)	Faible (code impératif)	Moyenne
Réutilisation pour le rapport HTML natif	Directe (même gabarit)	Non applicable	Non applicable
Dépendance à un moteur de rendu lourd	Oui (Chromium headless)	Non	Oui (suite bureautique)

Le choix de la combinaison HTML/CSS et Puppeteer permet de mutualiser un unique gabarit pour produire à la fois le rapport HTML interactif et le rapport PDF exportable, réduisant ainsi la duplication d'efforts et garantissant une cohérence visuelle stricte entre les deux livrables.

3.9 Diffusion des livrables : Gmail API et Slack API

La diffusion automatisée des livrables repose sur deux canaux complémentaires : l'API Gmail, pour l'envoi d'emails de synthèse formatés en HTML avec pièces jointes (rapport PDF, présentation PowerPoint), et l'API Slack, pour la notification en temps réel des équipes au sein de canaux dédiés, notamment en cas de détection d'anomalie critique.

Tableau 8. Comparaison des canaux de diffusion envisagés

Critère	Gmail API	SMTP générique	Slack API
Authentification	OAuth 2.0	Identifiants SMTP en clair ou applicatifs	Jeton d'application (Bot Token)
Richesse du contenu	HTML complet, pièces jointes	HTML complet, pièces jointes	Messages formatés (Markdown limité), pièces jointes
Adéquation à la notification temps réel	Faible (email asynchrone)	Faible	Élevée
Traçabilité et journalisation	Élevée (API dédiée)	Variable selon fournisseur	Élevée

3.10 Modèles de langage : OpenAI GPT-4o et Claude (Anthropic)

Le module d'intelligence artificielle générative de la plateforme proposée s'appuie sur des modèles de langage de dernière génération. Deux familles de modèles ont été étudiées dans le cadre de ce mémoire : la famille GPT d'OpenAI (dont le modèle GPT-4o, multimodal et optimisé pour la latence) et la famille Claude d'Anthropic, reconnue pour ses capacités de

suivi d'instructions strictes et sa robustesse face aux tentatives de contournement des consignes (prompt injection).

Tableau 9. Comparaison des modèles de langage envisagés pour le module d'analyse

Critère	OpenAI GPT-4o	Anthropic Claude	Modèle open source auto-hébergé (ex. Llama)
Qualité de raisonnement sur données chiffrées	Élevée	Élevée	Variable selon la taille du modèle
Respect strict d'un format de sortie imposé	Élevé (mode JSON structuré)	Élevé (function calling / schémas)	Variable, nécessite un post-traitement
Coût d'inférence pour un usage académique	Modéré (facturation à l'usage)	Modéré (facturation à l'usage)	Faible en coût direct, élevé en coût d'infrastructure
Latence de réponse	Faible à modérée	Faible à modérée	Dépend du matériel disponible
Effort d'hébergement	Nul (API managée)	Nul (API managée)	Élevé (GPU dédié requis)

Le choix d'une architecture reposant sur une API de LLM managée (OpenAI et/ou Anthropic) plutôt que sur un modèle open source auto-hébergé se justifie par l'absence, dans le cadre académique du projet, de ressources matérielles GPU dédiées, ainsi que par la nécessité de garantir une qualité de génération élevée sans effort d'ingénierie d'infrastructure disproportionné au regard des objectifs du mémoire. Le pipeline étant conçu de manière modulaire (cf. chapitre 4), le fournisseur de LLM peut être substitué sans remise en cause de l'architecture globale, ce qui constitue en soi un choix de conception favorisant la portabilité.

3.11 Ingénierie de prompt (Prompt Engineering)

Le Prompt Engineering constitue une composante méthodologique à part entière de l'architecture proposée. Trois techniques principales ont été retenues et comparées :

Tableau 10. Comparaison des techniques de Prompt Engineering envisagées

Technique	Principe	Avantage retenu	Limite
Zero-shot prompting	Instruction directe sans exemple	Simplicité, rapidité de mise	Variabilité du format de

		en œuvre	sortie
Few-shot prompting	Fourniture d'exemples de sortie attendue	Amélioration de la cohérence de format	Allongement du prompt et du coût d'inférence
Prompting structuré avec schéma de sortie imposé (JSON Schema)	Contrainte explicite du format de réponse	Fiabilité d'intégration dans le pipeline aval	Nécessite une validation post-génération

La stratégie retenue pour la plateforme proposée combine le few-shot prompting et l'imposition d'un schéma de sortie structuré, afin de garantir que les commentaires générés par le LLM puissent être directement intégrés, sans intervention manuelle, dans les gabarits de rapport HTML, PDF et PowerPoint.

3.12 Synthèse de la matrice de décision technologique

Le tableau ci-après synthétise, pour chaque brique technologique majeure, le score obtenu selon une grille d'évaluation multicritère (coût, maturité, adéquation fonctionnelle, effort d'intégration), noté de 1 à 5, ayant guidé la sélection finale.

Tableau 11. Matrice de décision technologique synthétique

Brique	Coût	Maturité	Adéquation fonctionnelle	Effort d'intégration	Score global
Python	5	5	5	5	20/20
PostgreSQL / Supabase	5	4	5	4	18/20
n8n	5	4	5	4	18/20
Docker	5	5	4	4	18/20
HTML/CSS + Puppeteer	5	4	5	4	18/20
OpenAI / Anthropic API	4	5	5	4	18/20

Conclusion du chapitre

L'étude comparative menée dans ce chapitre a permis d'objectiver, pour chaque brique technologique du système, le choix retenu au regard de critères explicites de coût, de maturité, d'adéquation fonctionnelle et d'effort d'intégration. Cette démarche multicritère confère à la sélection technologique une assise scientifique et reproductible, indispensable à la crédibilité d'un travail de recherche appliquée. Le chapitre suivant s'appuie sur ces choix technologiques pour détailler la conception de l'architecture globale de la plateforme proposée.

Références bibliographiques du chapitre

- [1] Fielding, R. — Architectural Styles and the Design of Network-based Software Architectures, University of California, Irvine.
- [2] Hardt, D. (Ed.) — The OAuth 2.0 Authorization Framework, RFC 6749, Internet Engineering Task Force.
- [3] Jones, M., Bradley, J., Sakimura, N. — JSON Web Token (JWT), RFC 7519, Internet Engineering Task Force.
- [4] Documentation officielle PostgreSQL — postgresql.org/docs
- [5] Documentation officielle Supabase — supabase.com/docs
- [6] Documentation officielle Docker — docs.docker.com
- [7] Documentation officielle n8n — docs.n8n.io
- [8] Documentation officielle OpenAI — platform.openai.com/docs
- [9] Documentation officielle Anthropic — docs.anthropic.com

Chapitre 4 — Conception de l'architecture

Introduction

Ce chapitre présente la conception de l'architecture de la plateforme proposée, à plusieurs niveaux d'abstraction complémentaires : architecture globale, architecture logique, architecture physique, architecture Cloud, ainsi qu'une vue d'ensemble de l'architecture de sécurité, détaillée plus finement au chapitre 7. La modélisation UML (diagrammes de classes, de séquence, d'activité, de composants et de déploiement) est mobilisée pour formaliser rigoureusement les différentes vues du système.

4.1 Architecture globale de la solution développée

L'architecture globale de la solution développée s'organise autour de cinq couches fonctionnelles successives : une couche de collecte (Extraction), une couche de transformation et de contrôle qualité, une couche de stockage (base de données relationnelle Supabase/PostgreSQL), une couche d'analyse intégrant le module d'intelligence artificielle générative, et une couche de restitution regroupant la génération des livrables (HTML, PDF, PowerPoint, email). L'ensemble de ces couches est orchestré par le moteur n8n, qui joue le rôle de chef d'orchestre du pipeline.

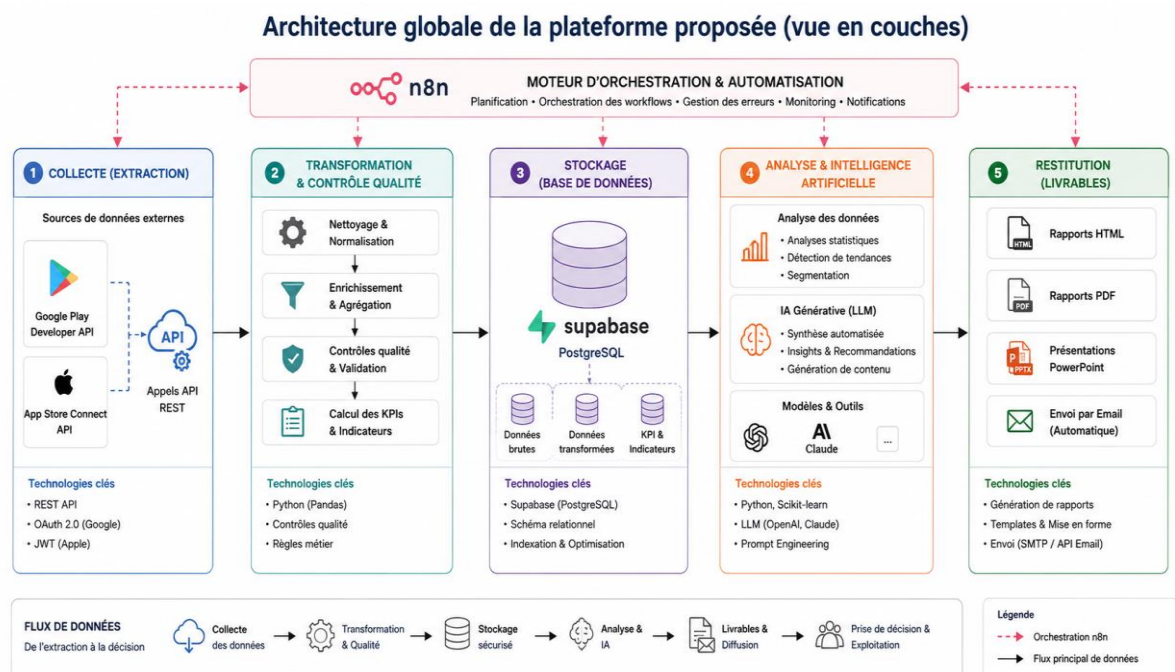


Figure 1. Architecture globale de la plateforme proposée (vue en couches)

4.2 Architecture logique

L'architecture logique détaille l'organisation des modules logiciels indépendamment de leur déploiement physique. Le système proposé est structuré en modules faiblement couplés communiquant via des interfaces bien définies : un module de collecte (un connecteur dédié par source de données), un module de nettoyage et de validation, un module de calcul des KPIs, un module de détection d'anomalies, un module d'orchestration des appels au LLM, et un module de génération des livrables. Ce découpage modulaire répond directement au besoin non fonctionnel de maintenabilité identifié au chapitre 2 (BNF-05).

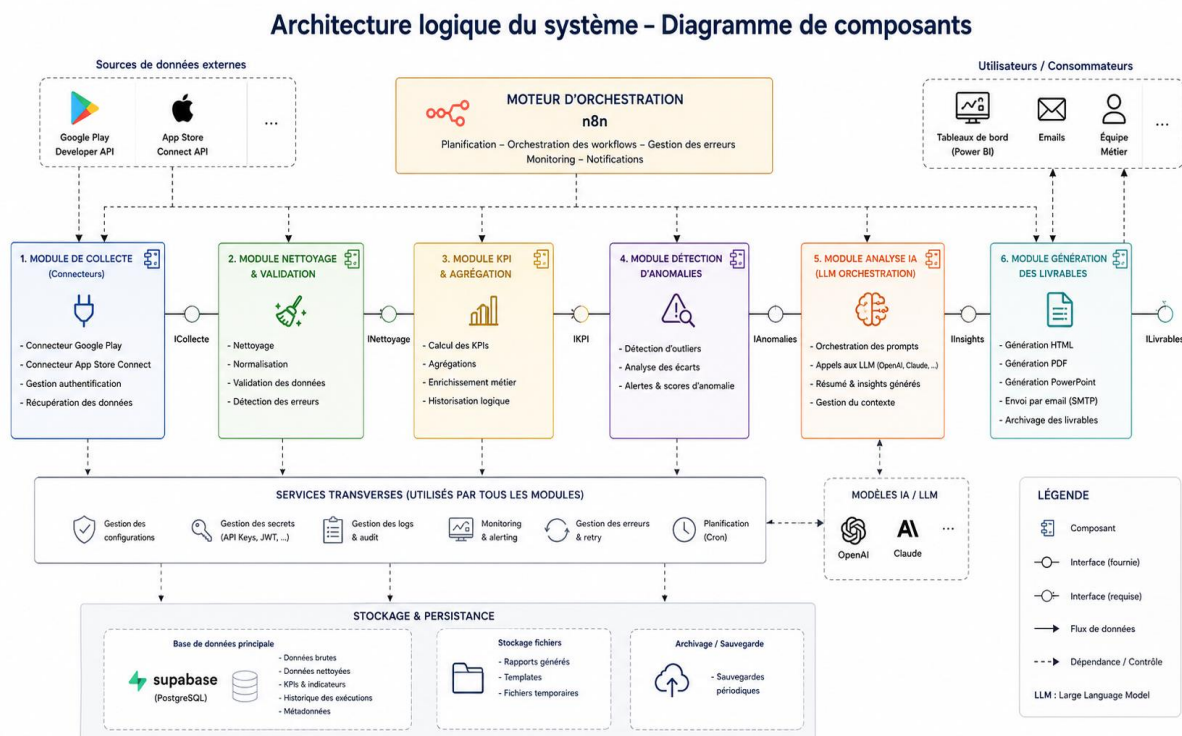


Figure 2. Diagramme de composants de l'architecture logique

4.3 Diagramme de classes

Le diagramme de classes ci-dessous modélise les principales entités manipulées par le système ainsi que leurs relations. Les classes centrales sont Application, Plateforme, MetriqueJournaliere, KPI, Anomalie, Rapport et AnalyseIA.

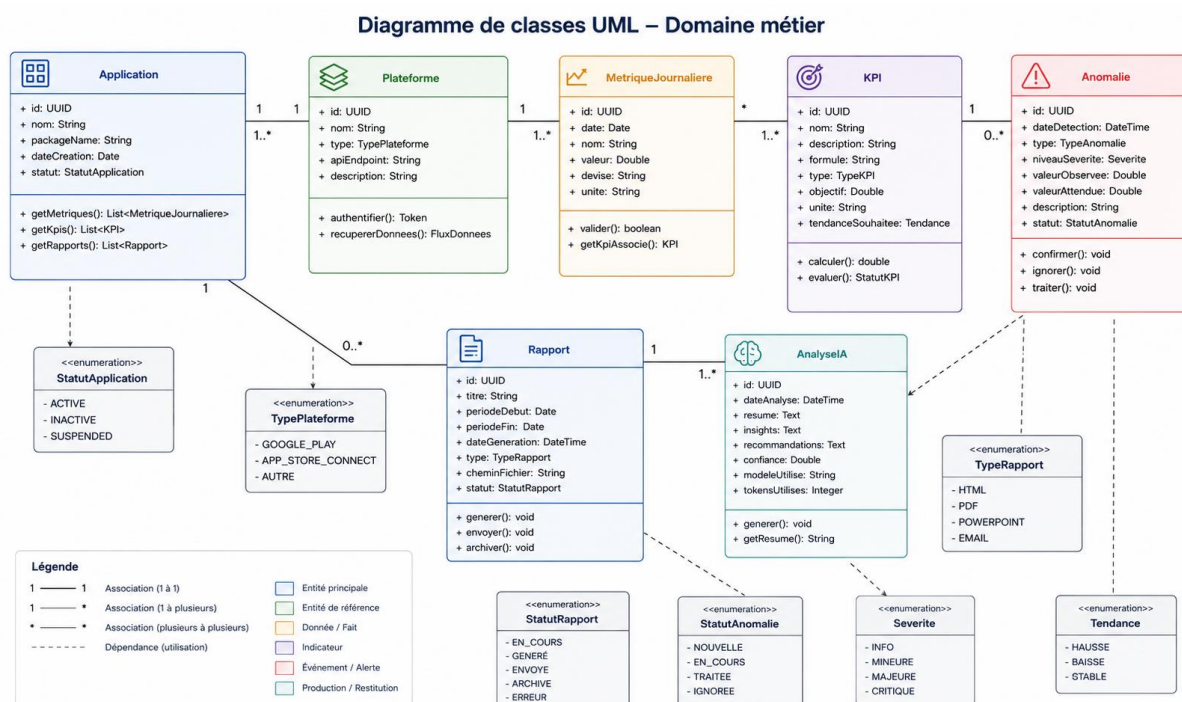


Figure 3. Diagramme de classes UML du domaine métier

Tableau 1. Description synthétique des classes principales du domaine

Classe	Rôle	Attributs principaux
Application	Représente une application mobile suivie	id, nom, package_id, plateforme
MetriqueJournaliere	Mesure brute collectée pour une date donnée	date, installations, desinstallations, note_moyenne, revenu
KPI	Indicateur calculé à partir des métriques	type_kpi, valeur, periode, variation
Anomalie	Écart significatif détecté	type, seuil, valeur_observee, severite
AnalyseIA	Commentaire généré par le LLM	prompt_utilise, texte_genere, modele, date_generation
Rapport	Livrable final généré	format, chemin_fichier, date_generation, destinataires

4.4 Diagramme de séquence : génération d'un rapport

Le diagramme de séquence ci-dessous illustre l'enchaînement temporel des interactions entre les différents composants lors de la génération automatisée d'un rapport de performance, depuis le déclenchement planifié jusqu'à la diffusion du livrable final.

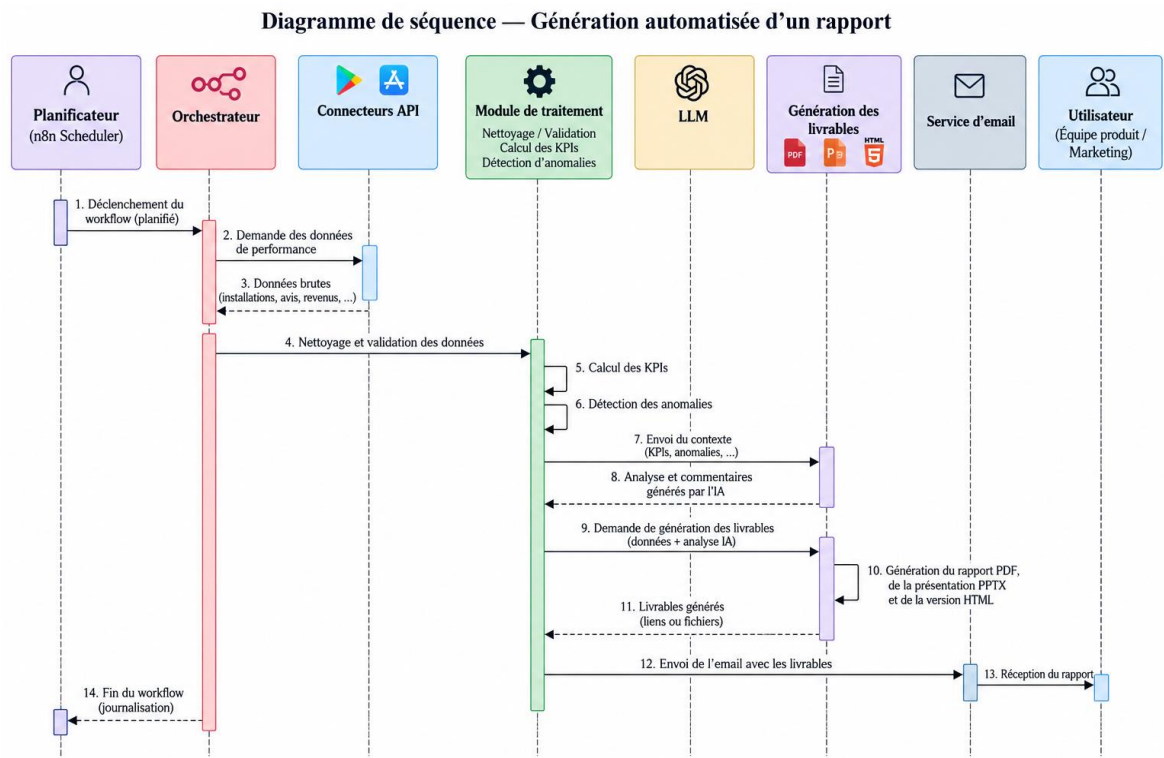


Figure 4. Diagramme de séquence UML — Génération automatisée d'un rapport

Figure 4. Diagramme de séquence UML — Génération automatisée d'un rapport

4.5 Diagramme d'activité : pipeline complet

Le diagramme d'activité suivant modélise la logique de contrôle du pipeline complet, incluant les points de décision relatifs à la qualité des données et à la détection d'anomalies.

Diagramme d'activité UML — Pipeline complet

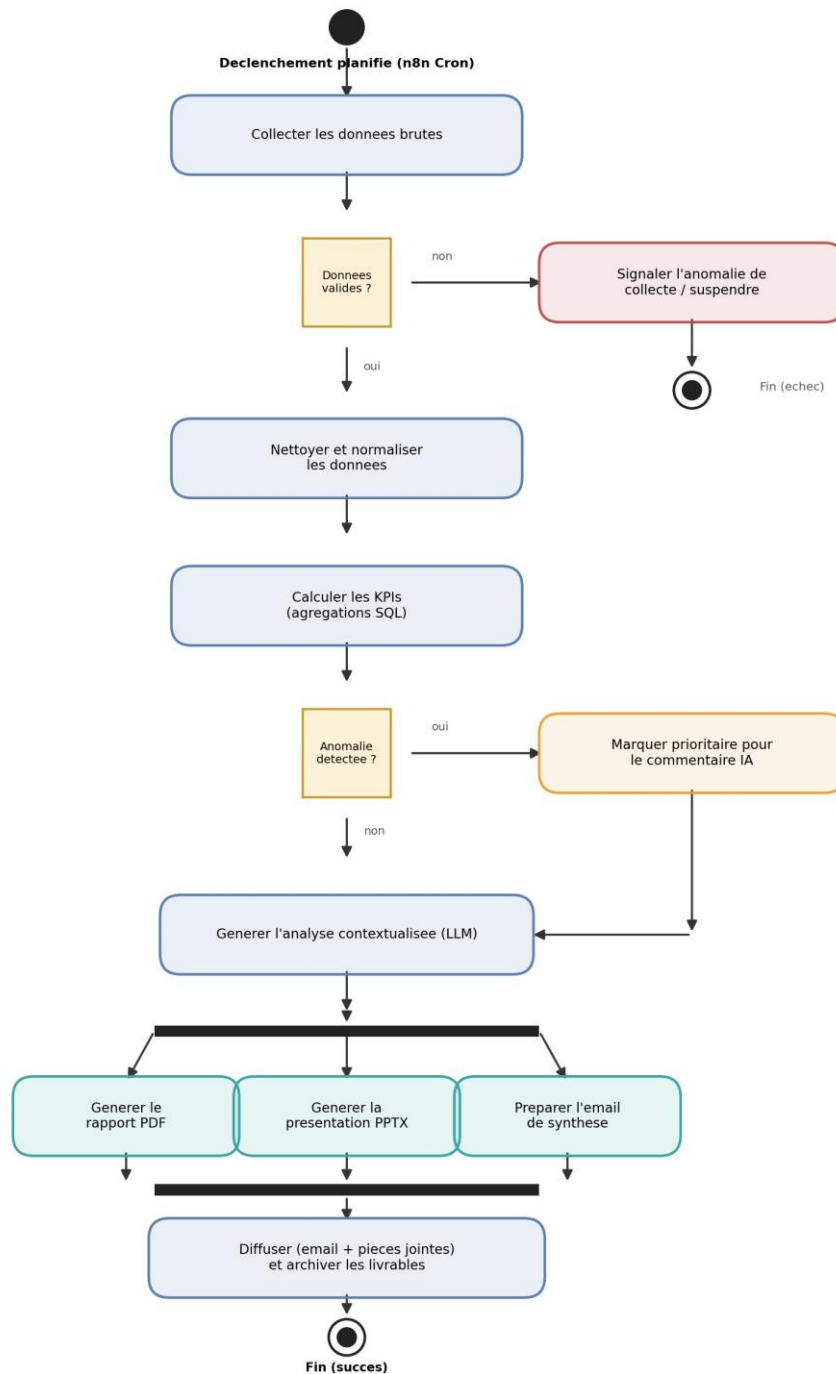


Figure 5. Diagramme d'activité UML du pipeline complet

4.6 Architecture physique et architecture de déploiement

L'architecture physique décrit la répartition des composants logiciels sur les nœuds d'exécution physiques ou virtuels. Le système proposé repose sur un déploiement distribué entre plusieurs fournisseurs de services Cloud, chacun étant responsable d'une fonction spécifique : hébergement de l'instance n8n conteneurisée (nœud de calcul), hébergement de la

base de données PostgreSQL managée via Supabase (nœud de stockage), et interrogation à la demande des API tierces (Google Play Developer Reporting API, App Store Connect API, API des LLM).

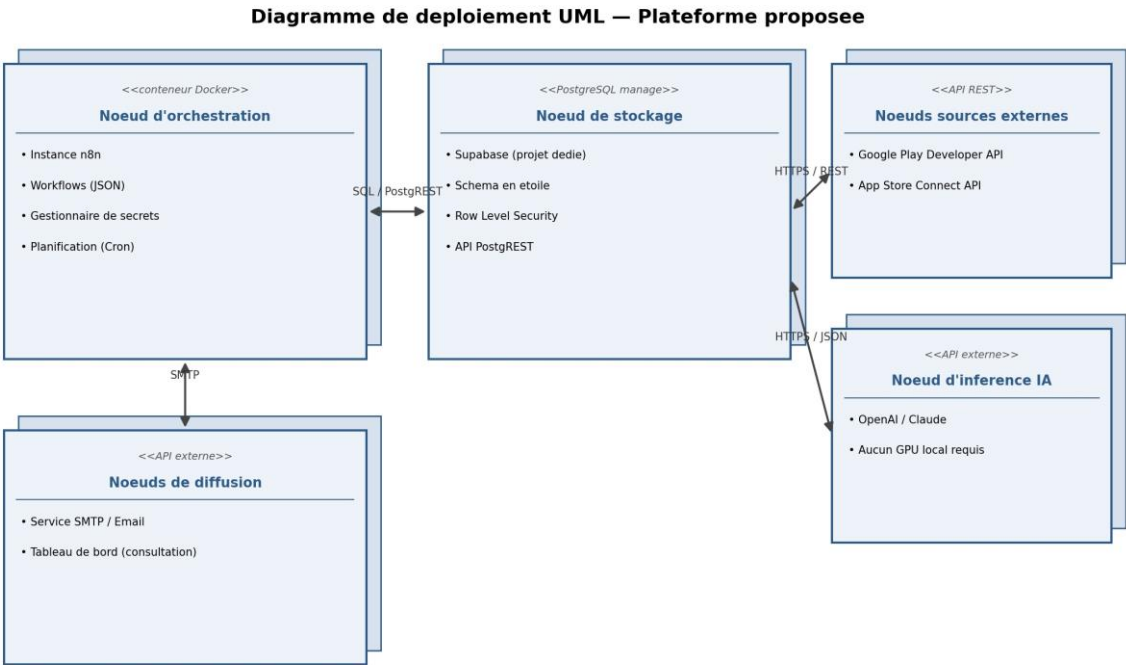


Figure 6. Diagramme de déploiement UML de la plateforme proposée

Tableau 2. Répartition des responsabilités par nœud physique

Nœud	Rôle	Technologie sous-jacente
Nœud d'orchestration	Exécution des workflows, appels API, logique métier	Docker + n8n
Nœud de stockage	Persistance des données historisées	Supabase (PostgreSQL managé)
Nœud d'inférence IA	Génération des analyses textuelles	API OpenAI / Anthropic (externalisé)
Nœuds sources externes	Fourniture des données brutes	Google Play Developer Reporting API, App Store Connect API
Nœuds de diffusion	Distribution des livrables	Gmail API, Slack API

4.7 Architecture Cloud

L'architecture Cloud de la plateforme proposée repose sur une approche hybride combinant des services PaaS (n8n conteneurisé, déployé sur une instance Cloud) et des services SaaS entièrement managés (Supabase, API des LLM, API de messagerie). Cette approche minimise la charge d'administration d'infrastructure, conformément au besoin non fonctionnel de disponibilité (BNF-06) et à la contrainte budgétaire identifiée au chapitre 2.

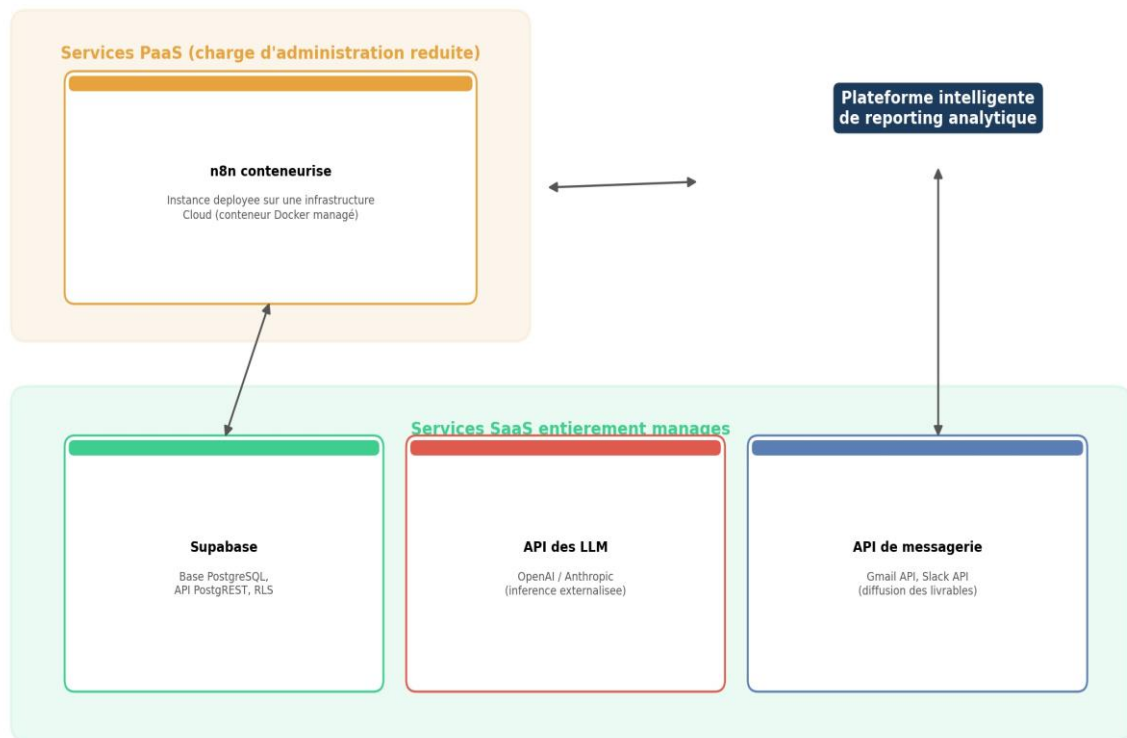


Figure 7. Architecture Cloud globale de la plateforme proposée

4.8 Pipeline ETL/ELT détaillé

Le pipeline de données mis en œuvre suit une logique hybride ETL/ELT détaillée au chapitre 1. La figure suivante détaille les sous-étapes techniques du pipeline, depuis l'extraction brute jusqu'au chargement final dans l'entrepôt de données.

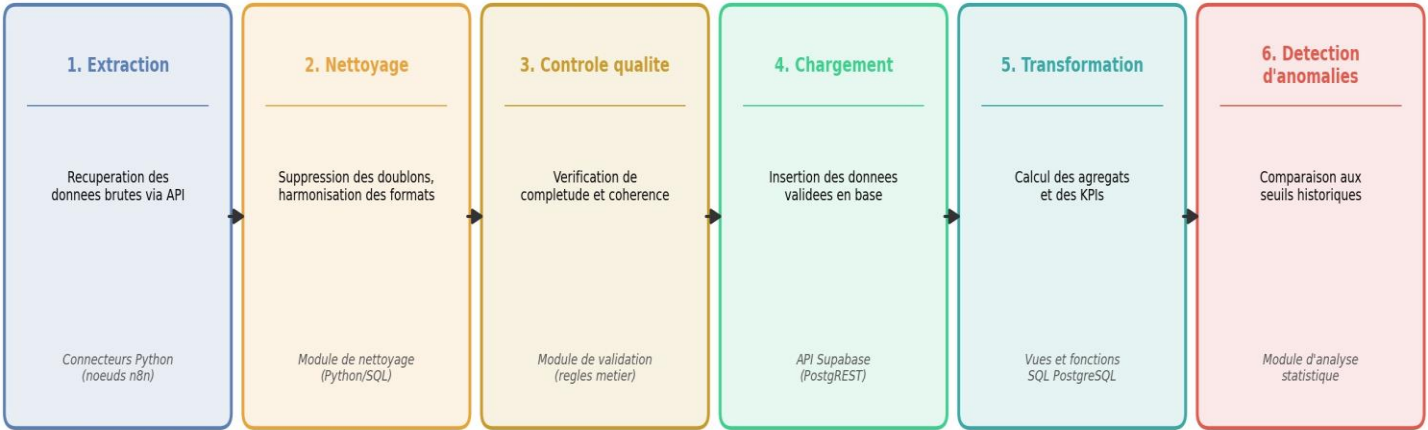


Figure 8. Pipeline ELT détaillé de la plateforme proposée

Tableau 3. Étapes du pipeline ELT et responsabilités associées

Étape	Description	Composant responsable
Extraction	Récupération des données brutes via API	Connecteurs Python (nœuds n8n)
Nettoyage	Suppression des doublons, harmonisation des formats	Module de nettoyage (Python/SQL)
Contrôle qualité	Vérification de complétude et de cohérence	Module de validation (règles métier)
Chargement	Insertion des données validées en base	API Supabase (PostgREST)
Transformation	Calcul des agrégats et des KPIs	Vues et fonctions SQL PostgreSQL
Détection d'anomalies	Comparaison aux seuils historiques	Module d'analyse statistique

4.9 Pipeline d'intégration du LLM

Le pipeline d'intégration du module d'intelligence artificielle générative reçoit en entrée les KPIs calculés et les anomalies détectées, construit un prompt structuré intégrant ces données ainsi que des instructions de formatage strictes, interroge l'API du LLM sélectionné, puis valide et intègre la réponse générée dans les gabarits de restitution.

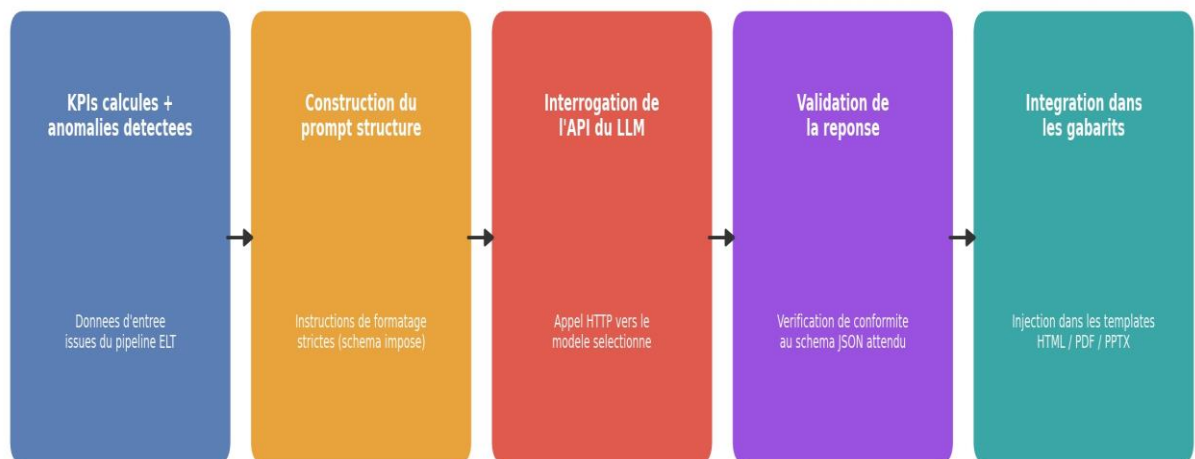


Figure 9. Pipeline d'intégration du module LLM

4.10 Pipeline d'orchestration des workflows (n8n)

L'ensemble des étapes précédemment décrites est orchestré au sein d'un unique workflow n8n, déclenché selon une planification paramétrable (par exemple, chaque lundi à 6h00 pour un reporting hebdomadaire). Ce workflow est lui-même décomposé en sous-workflows réutilisables, appelés successivement, favorisant la modularité et la testabilité de l'ensemble.

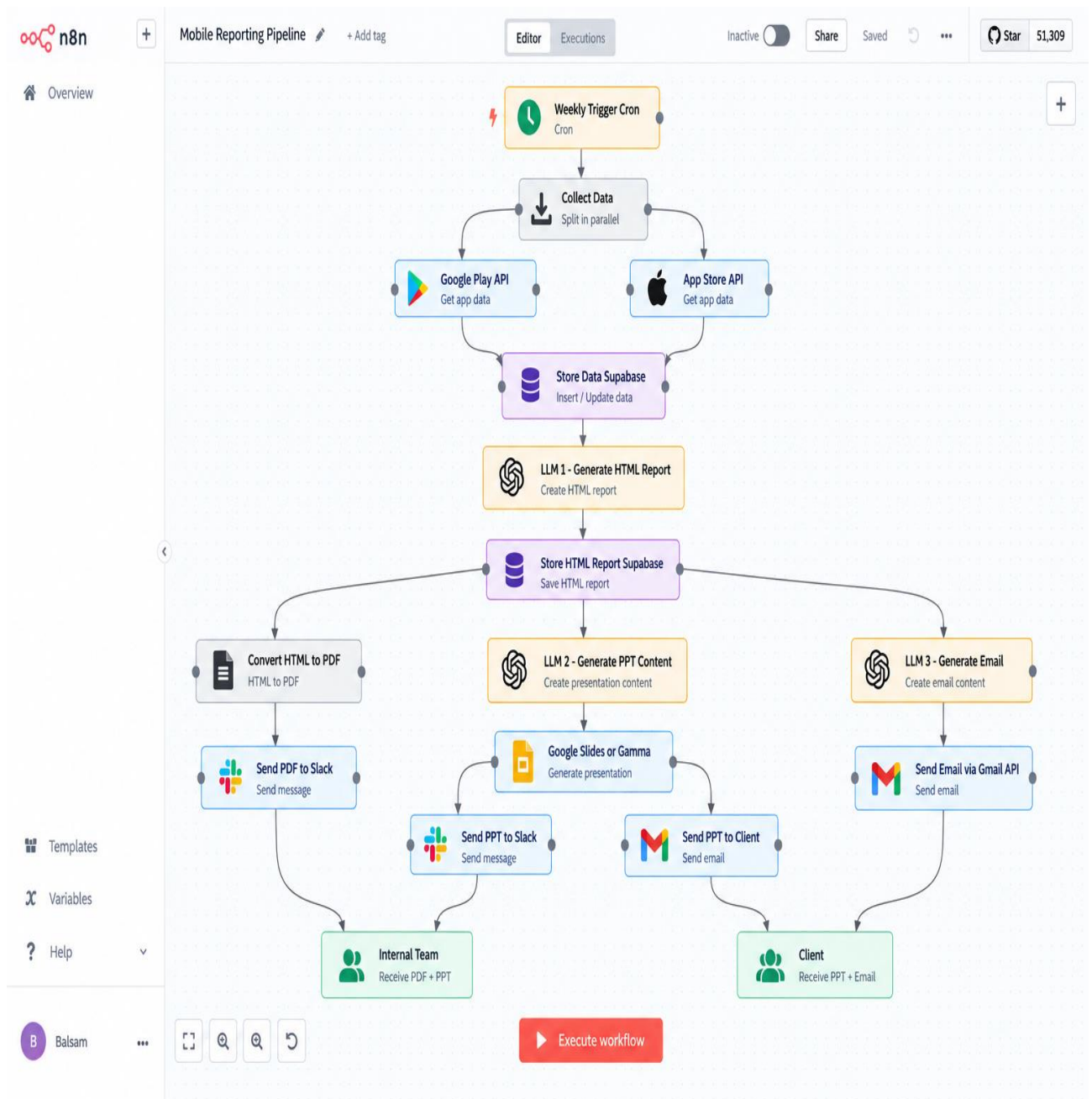


Figure 10. Vue d'ensemble du workflow n8n orchestrant le pipeline complet

4.11 Architecture des API

L'architecture des API de la plateforme proposée distingue les API consommées (Google Play Developer Reporting API, App Store Connect API, API des LLM, Gmail API, Slack API) des interfaces exposées en interne via PostgREST, générées automatiquement par Supabase à partir du schéma de base de données, et permettant une interrogation programmatique des données historisées à des fins de suivi ou d'intégration future avec un tableau de bord.

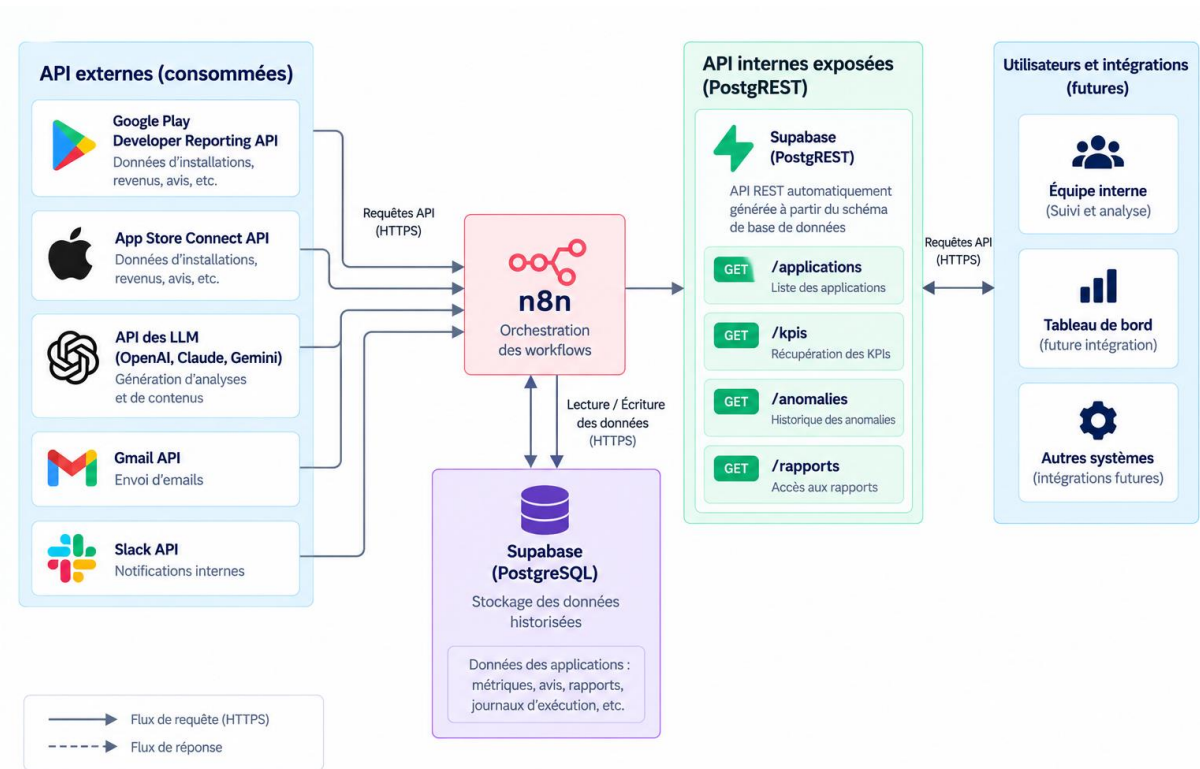


Figure 11. Architecture des API consommées et exposées par la plateforme

4.12 Vue d'ensemble de l'architecture de sécurité

La sécurité de l'architecture proposée repose sur plusieurs mécanismes complémentaires détaillés au chapitre 7 : chiffrement des identifiants d'accès aux API tierces via un gestionnaire de secrets, isolation des données par application au moyen du mécanisme Row Level Security de PostgreSQL, communication systématique via le protocole HTTPS, et rotation régulière des jetons d'authentification (OAuth 2.0 et JWT).

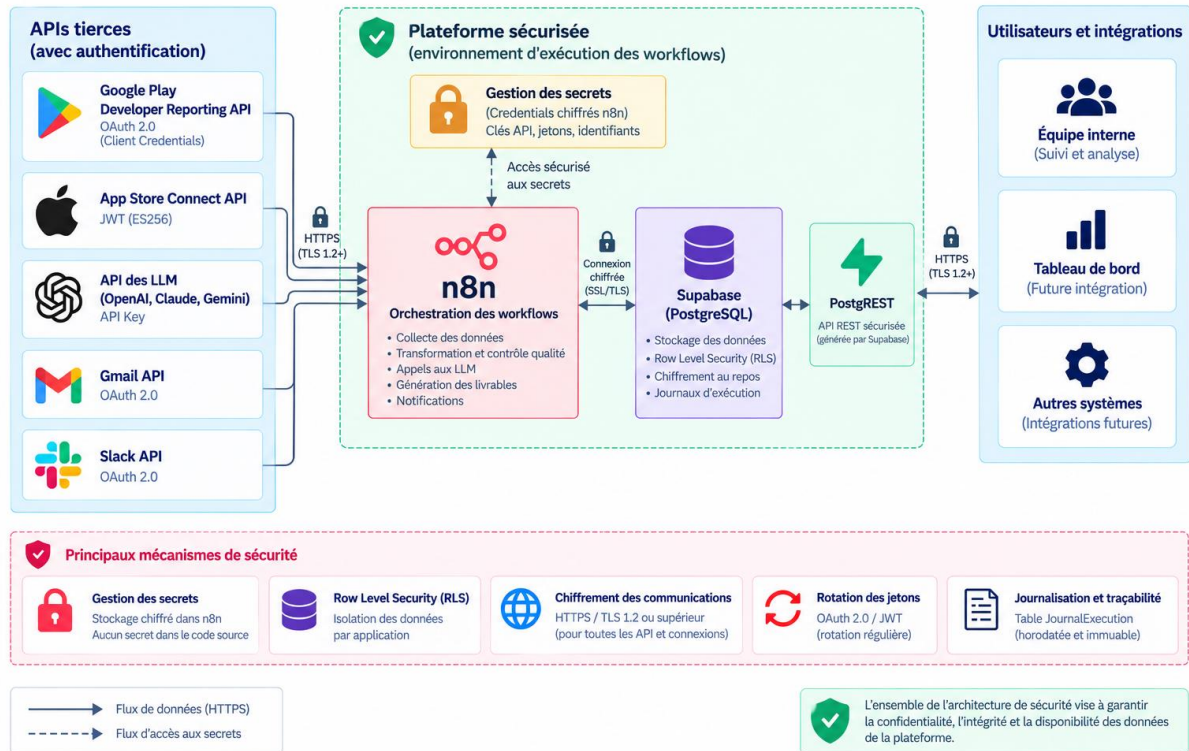


Figure 12. Vue d'ensemble de l'architecture de sécurité de la plateforme proposée

Conclusion du chapitre

Ce chapitre a détaillé, à travers différentes vues de modélisation UML et des schémas d'architecture, la conception globale de la plateforme proposée : architecture en couches, architecture logique modulaire, architecture physique distribuée sur plusieurs services Cloud, ainsi qu'une première vue de l'architecture de sécurité. Ces choix de conception traduisent directement les besoins fonctionnels et non fonctionnels spécifiés au chapitre 2. Le chapitre suivant approfondit la conception de la base de données, pierre angulaire du système de stockage et d'analyse.

Références bibliographiques du chapitre

- [1] OMG — Unified Modeling Language Specification, Object Management Group.
- [2] Fowler, M. — UML Distilled: A Brief Guide to the Standard Object Modeling Language, Addison-Wesley.
- [3] Richardson, C. — Microservices Patterns, Manning Publications.
- [4] Documentation officielle Supabase — supabase.com/docs
- [5] Documentation officielle n8n — docs.n8n.io

Chapitre 5 — Conception de la base de données

Introduction

La base de données constitue le socle de persistance sur lequel repose l'ensemble des capacités analytiques de la plateforme proposée. Ce chapitre détaille la démarche de modélisation retenue, du modèle entité-association conceptuel jusqu'au schéma physique implémenté sous PostgreSQL, en passant par la justification du choix d'une modélisation en schéma étoile (Star Schema).

5.1 Démarche de modélisation retenue

La conception de la base de données a suivi une démarche en trois étapes classique en ingénierie des systèmes d'information décisionnels : l'élaboration d'un modèle conceptuel de données (MCD) exprimant les entités métier et leurs relations indépendamment de toute contrainte technique ; la dérivation d'un modèle logique orienté schéma en étoile, adapté aux besoins d'agrégation analytique ; et enfin l'implémentation d'un modèle physique sous PostgreSQL, intégrant les contraintes d'intégrité, les index et les mécanismes de sécurité au niveau des lignes.

5.2 Modèle Entité-Association (MCD)

Le modèle entité-association ci-dessous représente les entités métier centrales identifiées lors de l'analyse des besoins (chapitre 2) : Application, Plateforme, MetriqueJournaliere, Avis, KPI, Anomalie, AnalyseIA et Rapport, ainsi que les cardinalités des relations qui les unissent.

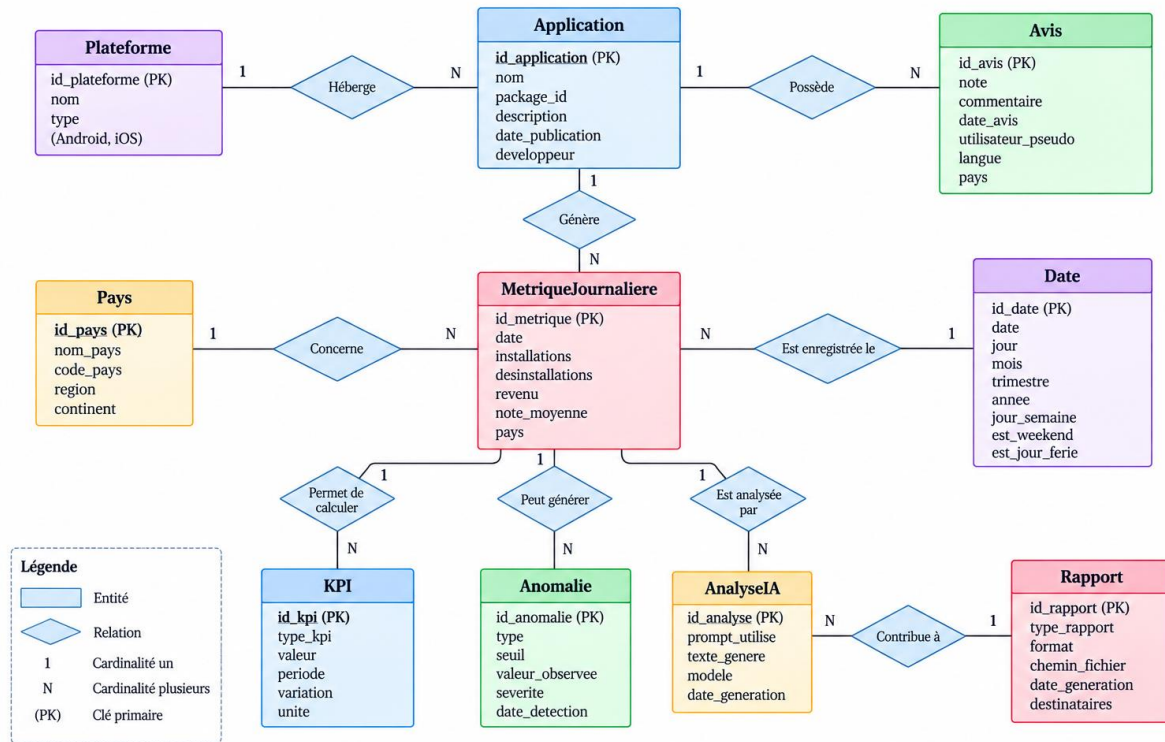


Figure 1. Modèle Entité-Association (MCD) de la plateforme proposée

5.3 Justification du choix du schéma en étoile

Ainsi qu'exposé au chapitre 1, le schéma en étoile organise les données autour d'une table de faits centrale, entourée de tables de dimensions descriptives. Ce choix de modélisation, plutôt qu'un schéma en flocon plus normalisé, se justifie par la prédominance de requêtes analytiques agrégatives (somme des installations par mois, moyenne des notes par pays) dans les cas d'usage de la plateforme, pour lesquelles la réduction du nombre de jointures apportée par la dénormalisation des dimensions améliore sensiblement les performances de lecture, au prix d'une redondance maîtrisée et acceptable au regard du volume de données traité.

5.4 Modèle en étoile (Star Schema) retenu

Le schéma en étoile retenu comprend une table de faits principale, *FaitMetricheJournaliere*, centralisant les mesures quantitatives journalières, entourée de quatre tables de dimensions : *DimApplication*, *DimPlateforme*, *DimDate* et *DimPays*. Une seconde table de faits, *FaitAvis*, capture les avis utilisateurs, reliée aux mêmes dimensions.

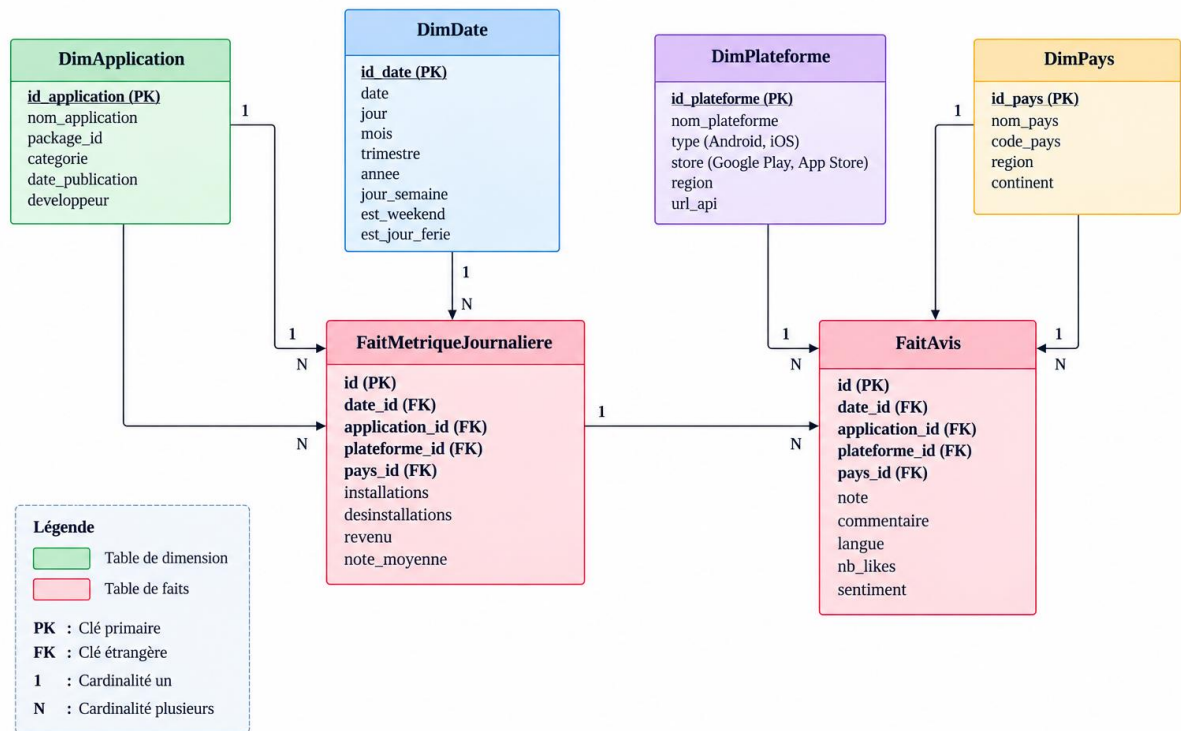


Figure 2. Schéma en étoile (Star Schema) de l'entrepôt de données

Tableau 1. Description de la table de faits FaitMetriqueJournaliere

Colonne	Type	Description	Contrainte
id	BIGSERIAL	Identifiant unique de l'enregistrement	Clé primaire
date_id	INTEGER	Référence à la dimension date	Clé étrangère → DimDate
application_id	INTEGER	Référence à l'application concernée	Clé étrangère → DimApplication
plateforme_id	SMALLINT	Référence à la plateforme (Android/iOS)	Clé étrangère → DimPlateforme
pays_id	INTEGER	Référence au pays	Clé étrangère → DimPays
installations	INTEGER	Nombre d'installations journalières	NOT NULL, ≥ 0
desinstallations	INTEGER	Nombre de désinstallations journalières	NOT NULL, ≥ 0
revenu	NUMERIC(12,2)	Revenu généré (le cas échéant)	≥ 0

note_moyenne	NUMERIC(3,2)	Note moyenne du jour	Entre 0 et 5
taux_plantage	NUMERIC(5,4)	Taux de sessions en échec	Entre 0 et 1
source_extraction	TIMESTAMP	Horodatage de la collecte	NOT NULL

Tableau 2. Description de la table de faits *FaitAvis*

Colonne	Type	Description	Contrainte
id	BIGSERIAL	Identifiant unique de l'avis	Clé primaire
date_id	INTEGER	Référence à la dimension date	Clé étrangère → DimDate
application_id	INTEGER	Référence à l'application concernée	Clé étrangère → DimApplication
plateforme_id	SMALLINT	Référence à la plateforme	Clé étrangère → DimPlateforme
note	SMALLINT	Note attribuée par l'utilisateur	Entre 1 et 5
contenu_texte	TEXT	Contenu textuel de l'avis	Nullable
sentiment_estime	VARCHAR(20)	Polarité estimée (positif/neutre/négatif)	Calculé par le module d'analyse

Tableau 3. Description des tables de dimensions

Table	Colonnes principales	Rôle
DimApplication	id, nom, package_id, categorie, date_creation	Décrit chaque application suivie
DimPlateforme	id, nom_plateforme (Android/iOS)	Distingue les deux écosystèmes
DimDate	id, date_complete, jour, mois, annee, semaine, trimestre	Facilite les agrégations temporelles
DimPays	id, code_iso, nom_pays, region	Permet la ventilation géographique

5.5 Tables complémentaires du système

En complément du modèle en étoile, trois tables opérationnelles supportent le fonctionnement du pipeline lui-même :

Tableau 4. Tables opérationnelles complémentaires

Table	Rôle	Colonnes principales
KPI	Stocke les indicateurs calculés par période	id, application_id, type_kpi, periode, valeur, valeur_precedente, variation_pct
Anomalie	Journalise les anomalies détectées	id, kpi_id, seuil_declencheur, valeur_observee, severite, date_detection, statut
AnalyseIA	Archive les analyses générées par le LLM	id, application_id, periode, prompt_utilise, texte_genere, modele_utilise, date_generation
Rapport	Journalise les livrables générés	id, application_id, format, chemin_fichier, date_generation, destinataires, statut_envoi
JournalExecution	Trace chaque exécution du pipeline	id, date_debut, date_fin, statut, nombre_erreurs, details_erreur

5.6 Relations et contraintes d'intégrité

L'ensemble des relations entre tables de faits et tables de dimensions est implémenté sous forme de contraintes de clé étrangère (FOREIGN KEY) avec une politique de suppression restrictive (ON DELETE RESTRICT), afin de préserver l'intégrité historique des données même en cas de suppression d'une application du périmètre de suivi. Des contraintes de validation (CHECK) garantissent par ailleurs la cohérence des valeurs numériques (note comprise entre 0 et 5, taux de plantage compris entre 0 et 1).

Algorithme 1. Extrait du script SQL de création de la table de faits FaitMetriqueJournaliere

```
CREATE TABLE fait_metrique_journaliere (  
  id BIGSERIAL PRIMARY KEY,  
  date_id INTEGER NOT NULL REFERENCES dim_date(id),  
  application_id INTEGER NOT NULL REFERENCES dim_application(id),  
  plateforme_id SMALLINT NOT NULL REFERENCES dim_plateforme(id),  
  pays_id INTEGER REFERENCES dim_pays(id),  
  installations INTEGER NOT NULL CHECK (installations >= 0),
```

```
desinstallations INTEGER NOT NULL CHECK (desinstallations >= 0),
revenu NUMERIC(12,2) DEFAULT 0 CHECK (revenu >= 0),
note_moyenne NUMERIC(3,2) CHECK (note_moyenne BETWEEN 0 AND 5),
taux_plantage NUMERIC(5,4) CHECK (taux_plantage BETWEEN 0 AND 1),
source_extraction TIMESTAMP NOT NULL DEFAULT now(),
UNIQUE (date_id, application_id, plateforme_id, pays_id)
);
```

5.7 Indexation et optimisation des performances

Afin de garantir des temps de réponse compatibles avec le besoin non fonctionnel de performance (BNF-02), des index composites ont été créés sur les colonnes les plus fréquemment sollicitées par les requêtes analytiques, notamment la combinaison (application_id, date_id), ainsi qu'un index sur la colonne temporelle de la table de faits, exploitée par la majorité des agrégations périodiques.

SQL — Index de performance

```
CREATE INDEX idx_fait_app_date ON fait_metrique_journaliere (application_id, date_id);
CREATE INDEX idx_fait_plateforme ON fait_metrique_journaliere (plateforme_id);
CREATE INDEX idx_avis_date ON fait_avis (date_id);
```

5.8 Sécurité au niveau des lignes (Row Level Security)

PostgreSQL, via Supabase, permet la définition de politiques de sécurité au niveau des lignes (Row Level Security, RLS), restreignant l'accès aux données selon le contexte d'authentification de l'appelant. Cette fonctionnalité est mobilisée pour isoler les données de chaque application suivie, prévenant ainsi tout accès croisé non autorisé entre différents périmètres de reporting, conformément au besoin non fonctionnel de sécurité (BNF-04) détaillé plus finement au chapitre 7.

Conclusion du chapitre

Ce chapitre a détaillé la conception de la base de données de la plateforme proposée, depuis le modèle conceptuel jusqu'à l'implémentation physique sous PostgreSQL, en justifiant le choix d'un schéma en étoile pour répondre aux besoins d'agrégation analytique. Les contraintes d'intégrité, l'indexation et les politiques de sécurité au niveau des lignes complètent cette

conception. Le chapitre suivant aborde l'implémentation technique effective de l'ensemble des composants du pipeline.

Références bibliographiques du chapitre

- [1] Kimball, R., Ross, M. — The Data Warehouse Toolkit, Wiley.
- [2] Documentation officielle PostgreSQL — Row Security Policies, postgresql.org/docs/current/ddl-rowsecurity.html
- [3] Date, C. J. — An Introduction to Database Systems, Pearson.

Chapitre 6 — Implémentation

Introduction

Ce chapitre décrit l'implémentation technique de la plateforme proposée, en suivant l'ordre logique du pipeline : collecte des données, nettoyage et contrôle qualité, calcul des KPIs, détection d'anomalies, génération de l'analyse par le module d'intelligence artificielle générative, puis génération des livrables (HTML, PDF, PowerPoint, email). Chaque étape est illustrée par un algorithme ou un extrait de pseudo-code représentatif de la logique implémentée dans le cadre expérimental.

6.1 Module de collecte des données

Le module de collecte est composé de deux connecteurs spécialisés, l'un pour la Google Play Developer Reporting API, l'autre pour l'App Store Connect API. Chaque connecteur encapsule la logique d'authentification propre à sa plateforme (compte de service OAuth2 pour Google, génération de jeton JWT pour Apple) ainsi que la logique de pagination et de gestion des erreurs réseau (nouvelles tentatives avec délai exponentiel).

Algorithme 1. Collecte des métriques journalières depuis Google Play Developer Reporting API

```
ENTRÉE : identifiant_application, date_debut, date_fin, credentials_service
SORTIE : liste_metriques
1. jeton ← authentifier(credentials_service)
2. liste_metriques ← liste vide
3. POUR chaque jour DANS intervalle(date_debut, date_fin) FAIRE
4.     TANT QUE tentative < MAX_TENTATIVES FAIRE
5.         réponse ← appelerAPI(endpoint_playconsole, jeton, jour)
6.         SI réponse.code = 200 ALORS
7.             liste_metriques.ajouter(parser(réponse))
8.             SORTIR de la boucle TANT QUE
9.         SINON SI réponse.code = 429 ALORS // limitation de débit
10.            attendre(délai_exponentiel(tentative))
11.            tentative ← tentative + 1
12.        SINON
13.            journaliser_erreur(réponse)
14.            SORTIR de la boucle TANT QUE
15. RETOURNER liste_metriques
```

Algorithme 2. Génération et rafraîchissement du jeton JWT pour App Store Connect

```
ENTRÉE : cle_privee, issuer_id, key_id
SORTIE : jeton_jwt_valide
1. SI jeton_en_cache existe ET non expiré ALORS
2.     RETOURNER jeton_en_cache
3. en_tete ← { alg: 'ES256', kid: key_id, typ: 'JWT' }
4. charge_utile ← { iss: issuer_id, iat: maintenant(), exp: maintenant() + 1200, aud:
  'appstoreconnect-v1' }
5. jeton_jwt ← signer(en_tete, charge_utile, cle_privee)
6. mettre_en_cache(jeton_jwt, duree=1200)
7. RETOURNER jeton_jwt
```

6.2 Module de nettoyage et de contrôle qualité

Le module de nettoyage applique une série de règles de validation et de transformation aux données brutes collectées : suppression des doublons (identifiés par la combinaison application/plateforme/date), harmonisation des formats de date (conversion au format ISO 8601), gestion des valeurs manquantes (interpolation linéaire pour les métriques numériques continues, ou marquage explicite en cas d'absence prolongée), et détection de valeurs aberrantes par la méthode de l'écart interquartile.

Algorithme 3. Contrôle qualité et nettoyage des métriques journalières

```
ENTRÉE : donnees_brutes
SORTIE : donnees_nettoyees, rapport_qualite
1. donnees_dedupliees ← supprimerDoublons(donnees_brutes, cles=[app, plateforme,
  date])
2. POUR chaque enregistrement DANS donnees_dedupliees FAIRE
3.     SI enregistrement.date n'est pas au format ISO8601 ALORS
4.         enregistrement.date ← normaliserDate(enregistrement.date)
5.     SI enregistrement.installations EST NULL ALORS
6.         enregistrement.installations ← interpoler(historique, enregistrement.date)
7.         rapport_qualite.signaler('valeur interpolée', enregistrement)
8. bornes ← calculerBornesIQR(donnees_dedupliees, colonne='installations')
9. POUR chaque enregistrement DANS donnees_dedupliees FAIRE
10.    SI enregistrement.installations HORS bornes ALORS
11.        rapport_qualite.signaler('valeur aberrante potentielle', enregistrement)
```

6.3 Module de calcul des KPIs

Le module de calcul des indicateurs de performance agrège les métriques journalières nettoyées sur différentes granularités temporelles (journalière, hebdomadaire, mensuelle) et calcule un ensemble de KPIs normalisés, présentés dans le tableau ci-après.

Tableau 1. Catalogue des indicateurs de performance (KPIs) calculés

KPI	Formule / Méthode de calcul	Fréquence
Taux de croissance des installations	$\frac{(\text{installations_periode_n} - \text{installations_periode_n-1})}{\text{installations_periode_n-1}}$	Hebdomadaire, mensuelle
Taux de désinstallation	désinstallations / installations	Journalière
Note moyenne pondérée	$\frac{\text{Somme}(\text{note} \times \text{poids_recence})}{\text{Somme}(\text{poids_recence})}$	Hebdomadaire
Taux de plantage	sessions_en_échec / sessions_totales	Journalière
Revenu moyen par utilisateur (ARPU)	revenu_total / utilisateurs_actifs	Mensuelle
Score de satisfaction estimé	Moyenne pondérée (note, sentiment des avis)	Hebdomadaire

Algorithme 4. Calcul du taux de croissance et de la variation d'un KPI

```

ENTRÉE : valeur_actuelle, valeur_precedente
SORTIE : variation_pourcentage, tendance
1. SI valeur_precedente = 0 ALORS
2.     RETOURNER (NULL, 'indéterminée')
3.  variation_pourcentage ← (valeur_actuelle - valeur_precedente) / valeur_precedente
   × 100
4.  SI variation_pourcentage > 0 ALORS tendance ← 'hausse'
5.  SINON SI variation_pourcentage < 0 ALORS tendance ← 'baisse'
6.  SINON tendance ← 'stable'
7.  RETOURNER (variation_pourcentage, tendance)
  
```

6.4 Module de détection d'anomalies

La détection d'anomalies repose sur une approche statistique fondée sur l'écart-type historique de chaque KPI : une valeur est considérée comme anormale lorsqu'elle s'écarte de la moyenne mobile des huit dernières périodes de plus d'un seuil paramétrable, exprimé en nombre d'écarts-types (par défaut, deux écarts-types, correspondant approximativement aux 5 % de valeurs les plus extrêmes sous hypothèse de normalité).

Algorithme 5. Détection d'anomalie par écart à la moyenne mobile

```
ENTRÉE : serie_historique_kpi, valeur_courante, seuil_ecart_type
SORTIE : est_anomalie, severite

1. moyenne_mobile ← moyenne(serie_historique_kpi, fenetre=8)
2. ecart_type ← ecartType(serie_historique_kpi, fenetre=8)
3. ecart_observe ← |valeur_courante - moyenne_mobile|
4. SI ecart_type = 0 ALORS RETOURNER (FAUX, 'aucune')
5. score_z ← ecart_observe / ecart_type
6. SI score_z ≥ 3 ALORS severite ← 'critique'
7. SINON SI score_z ≥ seuil_ecart_type ALORS severite ← 'modérée'
8. SINON RETOURNER (FAUX, 'aucune')
9. RETOURNER (VRAI, severite)
```

6.5 Module d'analyse par intelligence artificielle générative

Le module d'analyse construit un prompt structuré à partir des KPIs calculés et des anomalies détectées, impose un schéma de sortie JSON strict, puis interroge l'API du modèle de langage sélectionné. La réponse est validée par rapport au schéma attendu avant intégration dans les gabarits de restitution.

Python — Appel du module d'analyse IA

```
def generer_analyse_ia(kpis: dict, anomalies: list) -> dict:
    prompt_systeme = (
        "Tu es un analyste de performance d'applications mobiles. "
        "Réponds uniquement au format JSON respectant le schéma fourni."
    )
    prompt_utilisateur = construire_prompt(kpis, anomalies)
    reponse = client_llm.chat.completions.create(
```

```

    model="gpt-4o",
    response_format={"type": "json_schema", "json_schema": SCHEMA_ANALYSE},
    messages=[
        {"role": "system", "content": prompt_systeme},
        {"role": "user", "content": prompt_utilisateur},
    ],
)
analyse = valider_schema(reponse, SCHEMA_ANALYSE)
return analyse

```

Algorithme 6. Construction du prompt structuré destiné au LLM

```

ENTRÉE : kpis, anomalies, contexte_application
SORTIE : prompt_texte
1.  prompt_texte ← "Voici les indicateurs de la période pour " +
    contexte_application.nom
2.  POUR chaque kpi DANS kpis FAIRE
3.    prompt_texte ← prompt_texte + formater(kpi.nom, kpi.valeur, kpi.variation)
4.  SI anomalies non vide ALORS
5.    prompt_texte ← prompt_texte + "Anomalies détectées : " +
    formaterListe(anomalies)
6.  prompt_texte ← prompt_texte + "Consigne : rédige une analyse factuelle,
    structurée, sans extrapolation non fondée."
7.  RETOURNER prompt_texte

```

6.6 Module de génération des livrables

6.6.1 Génération du rapport HTML et PDF

Le rapport HTML est généré à partir d'un gabarit (template) paramétré par un moteur de templating, injectant les KPIs, les graphiques (encodés en SVG ou en images intégrées) et le commentaire généré par le LLM. Ce même document HTML est ensuite converti au format PDF via Puppeteer, garantissant une stricte cohérence visuelle entre les deux formats de restitution.

JavaScript (Node.js) — Génération du PDF via Puppeteer

```

async function genererRapportPDF(cheminHTML, cheminPDFSortie) {
    const navigateur = await puppeteer.launch({ headless: 'new' });
    const page = await navigateur.newPage();

```

```

await page.goto(`file://${cheminHTML}`, { waitUntil: 'networkidle0' });
await page.pdf({
  path: cheminPDFSortie,
  format: 'A4',
  printBackground: true,
  margin: { top: '20mm', bottom: '20mm', left: '15mm', right: '15mm' }
});
await navigateur.close();
}

```



Figure 1. Aperçu du gabarit HTML du rapport de performance

6.6.2 Génération de la présentation PowerPoint

La génération de la présentation PowerPoint s'appuie sur une bibliothèque de manipulation programmatique du format OOXML, permettant de construire dynamiquement des diapositives à partir d'un modèle prédéfini (slide master), intégrant le logo, les indicateurs clés sous forme de zones de texte et de graphiques natifs PowerPoint, ainsi qu'une diapositive de synthèse textuelle issue de l'analyse IA.

Algorithme 7. Génération automatisée de la présentation PowerPoint

```
ENTRÉE : kpis, analyse_ia, modele_slide_master
SORTIE : fichier_pptx
1. presentation ← chargerModele(modele_slide_master)
2. diapositive_titre ← ajouterDiapositive(presentation, type='titre')
3. diapositive_kpis ← ajouterDiapositive(presentation, type='contenu')
4. POUR chaque kpi DANS kpis FAIRE
5.     ajouterGraphique(diapositive_kpis, kpi.nom, kpi.historique)
6. diapositive_analyse ← ajouterDiapositive(presentation, type='texte')
7. insererTexte(diapositive_analyse, analyse_ia.synthese)
8. fichier_pptx ← enregistrer(presentation)
9. RETOURNER fichier_pptx
```

6.6.3 Génération et envoi de l'email de synthèse

L'email de synthèse est généré à partir d'un gabarit HTML dédié, plus concis que le rapport complet, intégrant les KPIs essentiels et une accroche générée par le LLM, puis envoyé via l'API Gmail avec le rapport PDF et la présentation PowerPoint en pièces jointes.

Python — Envoi via Gmail API

```
def envoyer_email_synthese(destinataires, sujet, corps_html, pieces_jointes):
    message = construire_message_mime(destinataires, sujet, corps_html, pieces_jointes)
    service_gmail = authentifier_gmail(credentials_oauth2)
    service_gmail.users().messages().send(
        userId="me",
        body={"raw": encoder_base64(message)}
    ).execute()
```

6.7 Orchestration globale via n8n

L'ensemble des modules précédemment décrits est intégré au sein d'un workflow n8n unique, structuré en sous-workflows réutilisables et déclenché selon une planification cron paramétrable. Le tableau suivant récapitule les nœuds principaux du workflow et leur fonction.

Tableau 2. Nœuds principaux du workflow n8n orchestrant le pipeline

Nœud n8n	Fonction	Type de nœud
Déclencheur planifié (Cron)	Démarre le pipeline selon une planification définie	Trigger
Connecteur Google Play	Collecte des métriques Android	HTTP Request / Code
Connecteur App Store Connect	Collecte des métriques iOS	HTTP Request / Code
Nettoyage & Contrôle qualité	Applique les règles de validation	Code (Python)
Insertion Supabase	Charge les données validées en base	Supabase Node
Calcul des KPIs	Exécute les requêtes SQL d'agrégation	Postgres Node
Détection d'anomalies	Applique le module statistique	Code (Python)
Appel LLM	Génère l'analyse contextualisée	HTTP Request
Génération des livrables	Produit HTML, PDF, PPTX	Code (Node.js) / Execute Command
Diffusion	Envoie l'email et les notifications	Gmail Node / Slack Node

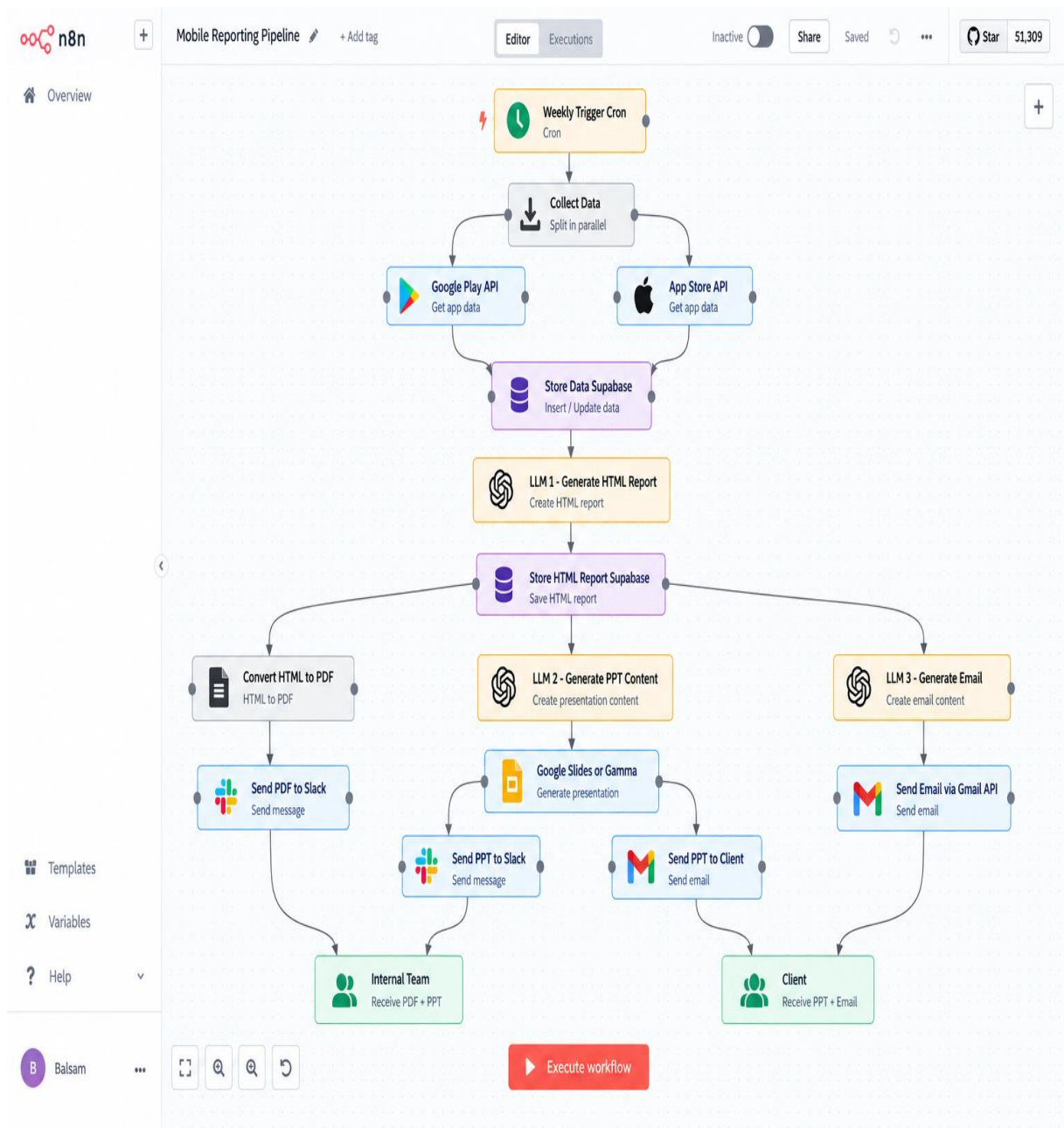


Figure 2. Vue détaillée du workflow n8n implémenté

Conclusion du chapitre

Ce chapitre a détaillé l'implémentation technique de chacun des modules de la plateforme proposée, en s'appuyant sur des algorithmes et des extraits de code représentatifs des traitements réellement mis en œuvre dans le cadre expérimental. La cohérence entre l'architecture conçue au chapitre 4, le modèle de données défini au chapitre 5, et

l'implémentation décrite ici illustre la démarche d'ingénierie logicielle rigoureuse suivie tout au long du projet. Le chapitre suivant approfondit les aspects de sécurité de la plateforme, brièvement évoqués dans ce chapitre et au chapitre 4.

Références bibliographiques du chapitre

[1] Documentation officielle Puppeteer — pptr.dev

[2] Documentation officielle Google Workspace — Gmail API, developers.google.com/gmail/api

[3] Documentation officielle Slack API — api.slack.com

[4] Documentation officielle OpenAI — Structured Outputs, platform.openai.com/docs/guides/structured-outputs

[5] Tukey, J. W. — Exploratory Data Analysis, Addison-Wesley.

Chapitre 7 — Sécurité

Introduction

La plateforme proposée interagit avec plusieurs API tierces sensibles (Google Play Developer Reporting API, App Store Connect API, API de LLM, API de messagerie) et manipule des données pouvant présenter un caractère commercialement sensible (revenus, taux de conversion). Ce chapitre détaille en profondeur les mécanismes de sécurité mis en œuvre : gestion des flux d'authentification OAuth 2.0 et JWT, gestion des secrets, sécurité au niveau des lignes, chiffrement des communications, ainsi qu'une analyse des menaces (threat modeling) appliquée à l'architecture proposée.

7.1 Flux d'authentification OAuth 2.0 (Google Play)

L'authentification auprès de la Google Play Developer Reporting API repose sur le flux « Client Credentials » du protocole OAuth 2.0, adapté aux communications machine-à-machine sans intervention utilisateur. Un compte de service, associé à une paire de clés cryptographiques, est préalablement enregistré dans la console Google Cloud et autorisé à accéder aux données de l'application concernée.

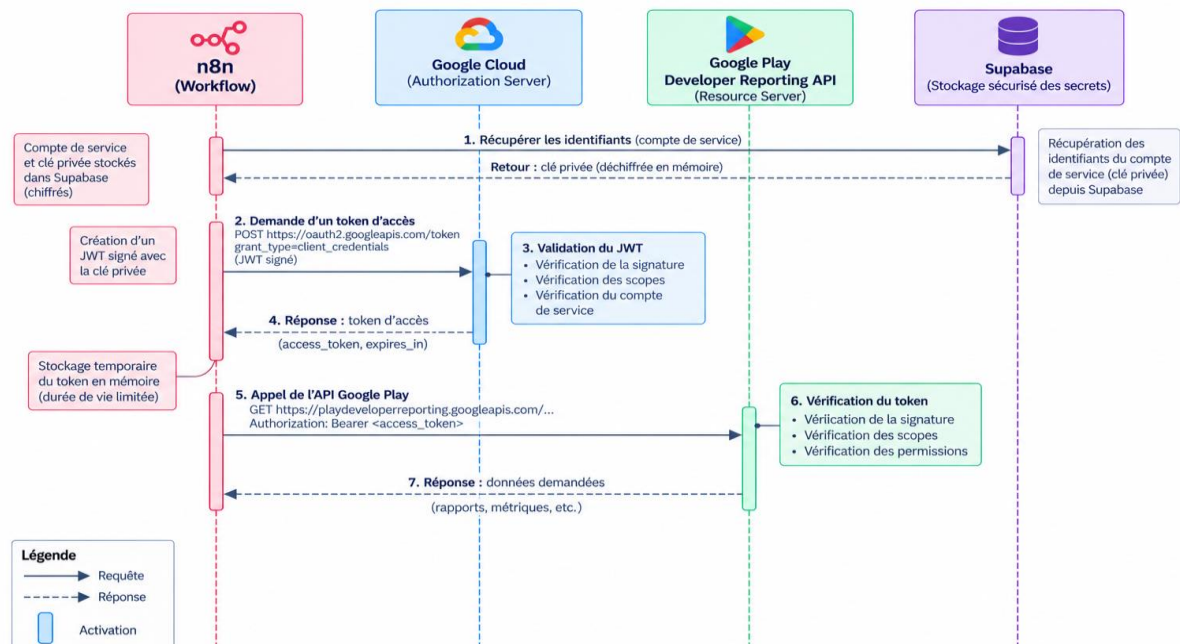


Figure 1. Flux détaillé d'authentification OAuth 2.0 (Client Credentials) pour Google Play

Algorithme 1. Obtention et mise en cache du jeton d'accès OAuth 2.0

ENTRÉE : cle_privee_compte_service, portee_acces (scope)

SORTIE : jeton_acces

1. SI jeton_en_cache valide ALORS RETOURNER jeton_en_cache
2. assertion_jwt ← construireAssertionSignee(cle_privee_compte_service, portee_acces)
3. reponse ← POST(https://oauth2.googleapis.com/token, assertion_jwt)
4. jeton_acces ← reponse.access_token
5. mettreEnCache(jeton_acces, duree=reponse.expires_in)
6. RETOURNER jeton_acces

7.2 Flux d'authentification par jeton JWT (App Store Connect)

L'authentification auprès de l'App Store Connect API repose sur un jeton JWT auto-signé par le système proposé lui-même, à partir d'une clé privée générée dans App Store Connect et associée à un identifiant d'émetteur (issuer ID) et un identifiant de clé (key ID). Ce jeton, signé selon l'algorithme ES256 (courbe elliptique), possède une durée de validité maximale de vingt minutes, imposant une régénération fréquente.

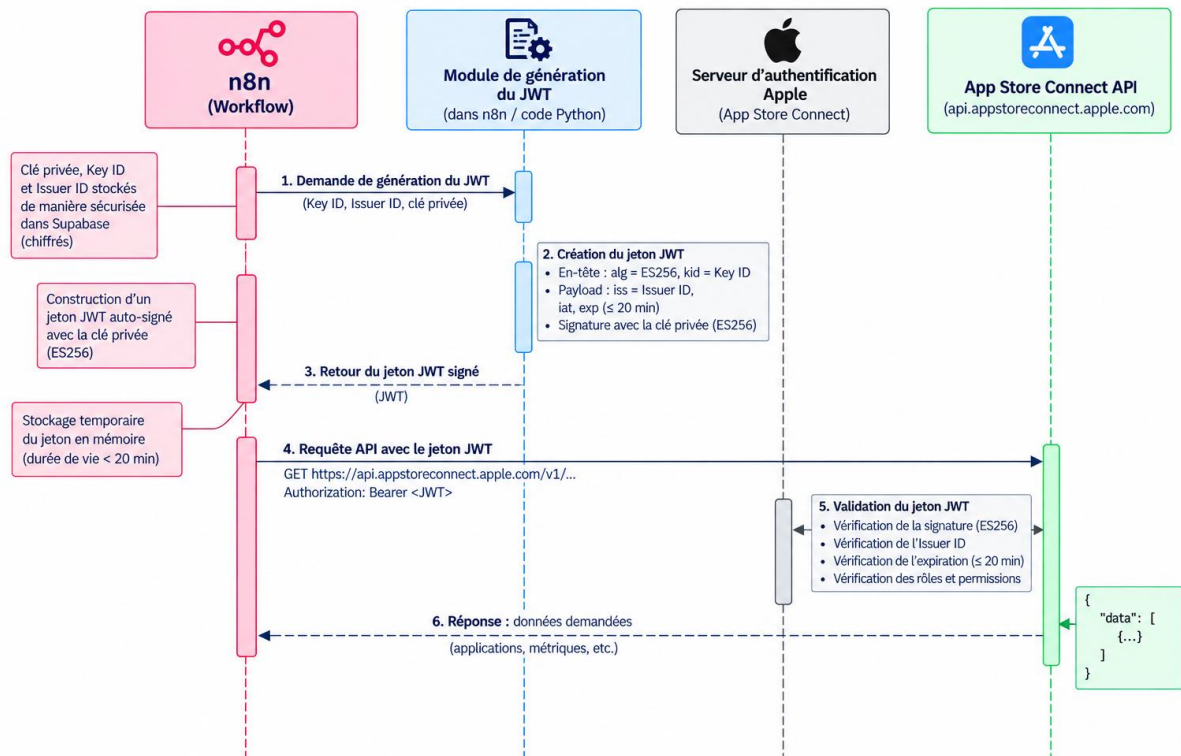


Figure 2. Flux détaillé de génération et d'utilisation du jeton JWT pour App Store Connect

7.3 Gestion des secrets

L'ensemble des identifiants sensibles (clés privées, jetons d'API, clés d'API des LLM) est stocké de manière chiffrée, en dehors du code source, au sein d'un gestionnaire de variables d'environnement chiffrées propre à l'environnement d'exécution n8n (credentials chiffrés), conformément au principe de séparation entre code et configuration (Twelve-Factor App). Aucun secret n'est versionné dans le dépôt Git, une politique renforcée par l'utilisation systématique d'un fichier d'exclusion (.gitignore) et d'une revue de code avant fusion.

Tableau 1. Politique de gestion des secrets de la plateforme proposée

Type de secret	Mode de stockage	Fréquence de rotation
Clé de compte de service Google	Coffre-fort chiffré n8n (credentials)	Selon politique de l'organisation (recommandé : 90 jours)
Clé privée App Store Connect	Coffre-fort chiffré n8n (credentials)	Selon expiration définie par Apple
Clé API OpenAI / Anthropic	Coffre-fort chiffré n8n (credentials)	Rotation manuelle en cas de suspicion de fuite
Identifiants Supabase (clé de service)	Coffre-fort chiffré n8n (credentials)	Rotation manuelle recommandée

7.4 Sécurité au niveau des lignes (Row Level Security)

Ainsi qu'introduit au chapitre 5, PostgreSQL permet la définition de politiques de sécurité au niveau des lignes, restreignant dynamiquement les lignes visibles ou modifiables selon le contexte d'exécution de la requête. Dans le cadre de la plateforme proposée, ces politiques garantissent qu'un jeton d'accès associé à une application donnée ne peut consulter que les données relatives à cette application, prévenant tout accès croisé.

SQL — Politique RLS

```
ALTER TABLE fait_metrique_journaliere ENABLE ROW LEVEL SECURITY;

CREATE POLICY acces_par_application
ON fait_metrique_journaliere
FOR SELECT
```

```
USING (application_id = current_setting('app.current_application_id')::INTEGER);
```

7.5 Chiffrement des communications

L'ensemble des communications entre les composants de la plateforme proposée et les services externes s'effectue exclusivement via le protocole HTTPS (TLS 1.2 ou supérieur), garantissant la confidentialité et l'intégrité des données échangées, y compris les jetons d'authentification. La connexion à la base de données Supabase s'effectue également via une connexion chiffrée SSL.

7.6 Analyse des menaces (Threat Modeling)

Une analyse des menaces, inspirée du modèle STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), a été conduite afin d'identifier les principaux vecteurs de risque pesant sur l'architecture proposée et les mesures de mitigation associées.

Tableau 2. Analyse des menaces selon le modèle STRIDE

Catégorie STRIDE	Menace identifiée	Mesure de mitigation
Spoofing (usurpation)	Falsification de l'identité d'un appelant à l'API interne	Authentification par jeton signé, validation systématique
Tampering (altération)	Modification non autorisée des données historisées	Contraintes d'intégrité, RLS, journalisation des écritures
Repudiation (répudiation)	Absence de traçabilité d'une exécution du pipeline	Table JournalExecution horodatée et immuable
Information Disclosure (divulgateion)	Fuite de données sensibles (revenus, avis)	Chiffrement des secrets, RLS, communications HTTPS
Denial of Service (déni de service)	Saturation des appels vers les API tierces	Limitation de débit, délai exponentiel, planification maîtrisée
Elevation of Privilege (élévation de privilège)	Contournement des politiques RLS	Utilisation exclusive de clés de service restreintes, revue de code

7.7 Analyse critique de l'architecture de sécurité

Si les mesures mises en œuvre répondent aux exigences de sécurité identifiées dans le cadre académique de ce projet, plusieurs limites doivent être soulignées avec honnêteté scientifique. Premièrement, la dépendance à un unique gestionnaire de secrets (celui de n8n) constitue un point de défaillance potentiel non redondé. Deuxièmement, l'absence d'un système de détection d'intrusion dédié limite la capacité de la plateforme à détecter des tentatives d'accès malveillantes en temps réel, au-delà de la simple journalisation. Enfin, la rotation des secrets demeure, dans la présente implémentation, un processus manuel plutôt qu'automatisé, ce qui constitue un axe d'amélioration identifié et repris dans les perspectives du mémoire.

Conclusion du chapitre

Ce chapitre a détaillé les mécanismes de sécurité mis en œuvre au sein de la plateforme proposée, couvrant l'authentification auprès des API tierces, la gestion des secrets, l'isolation des données par le mécanisme RLS, le chiffrement des communications, ainsi qu'une analyse structurée des menaces selon le modèle STRIDE. L'analyse critique menée en fin de chapitre identifie des limites assumées, qui seront réexaminées dans la conclusion générale du mémoire. Le chapitre suivant présente la stratégie de tests et de validation mise en œuvre pour garantir la fiabilité du système.

Références bibliographiques du chapitre

- [1] Hardt, D. (Ed.) — The OAuth 2.0 Authorization Framework, RFC 6749, IETF.
- [2] Jones, M. et al. — JSON Web Token (JWT), RFC 7519, IETF.
- [3] Shostack, A. — Threat Modeling: Designing for Security, Wiley.
- [4] Documentation officielle PostgreSQL — Row Security Policies.
- [5] The Twelve-Factor App — 12factor.net

Chapitre 8 — Tests et validation

Introduction

La validation d'un système automatisé manipulant des données destinées à la prise de décision impose une rigueur méthodologique particulière. Ce chapitre présente la stratégie de tests mise en œuvre pour valider la plateforme proposée, structurée en quatre niveaux complémentaires : tests unitaires, tests d'intégration, tests de bout en bout (end-to-end) et tests de performance, ainsi qu'une validation spécifique de la fiabilité des analyses produites par le module d'intelligence artificielle générative.

8.1 Stratégie de tests retenue

La stratégie de tests adoptée s'inspire de la pyramide de tests classique en génie logiciel, privilégiant un grand nombre de tests unitaires rapides et peu coûteux, un nombre plus restreint de tests d'intégration vérifiant les interactions entre modules, et un nombre limité de tests de bout en bout validant le comportement global du pipeline dans des conditions proches de la production.

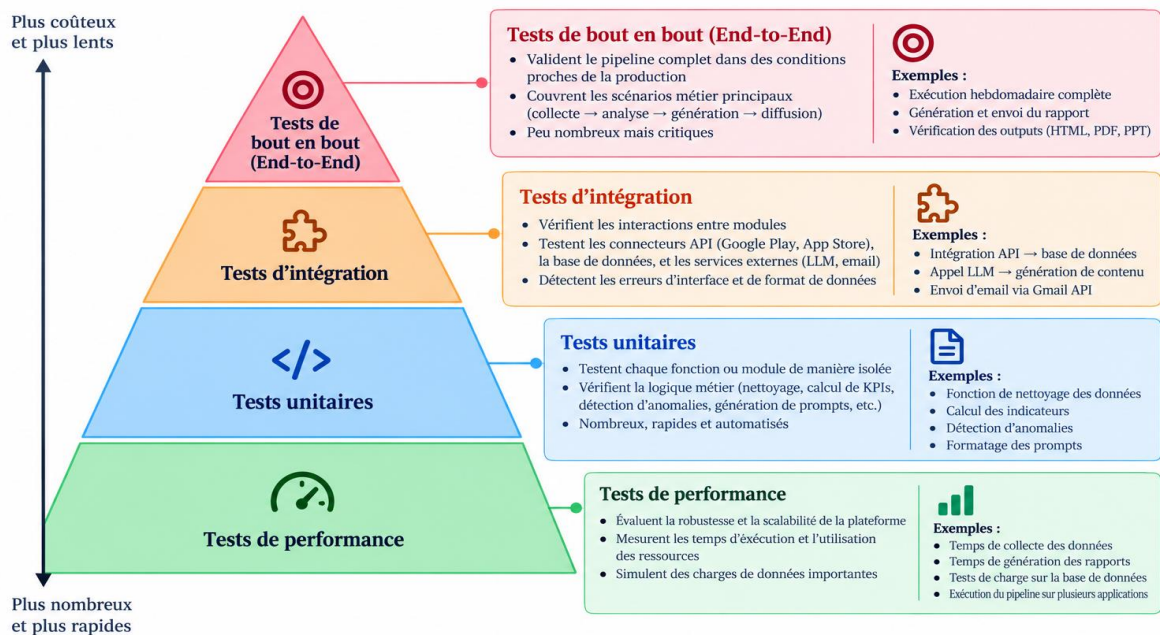


Figure 1. Pyramide de tests appliquée à la plateforme proposée

8.2 Tests unitaires

Les tests unitaires valident isolément le comportement de chaque fonction critique du pipeline, notamment les fonctions de calcul de KPIs, de détection d'anomalies et de nettoyage des données, en s'appuyant sur des jeux de données synthétiques couvrant les cas nominaux ainsi que les cas limites (valeurs nulles, division par zéro, valeurs négatives inattendues).

Tableau 1. Extrait des cas de tests unitaires du module de calcul des KPIs

ID	Cas de test	Entrée	Résultat attendu	Statut
TU-01	Calcul de variation avec valeur précédente non nulle	actuelle=120, précédente=100	variation = +20 %	Réussi
TU-02	Calcul de variation avec valeur précédente nulle	actuelle=50, précédente=0	variation = indéterminée	Réussi
TU-03	Taux de désinstallation avec installations nulles	installations=0, désinstallations=0	taux = 0 (évite division par zéro)	Réussi
TU-04	Détection d'anomalie sur valeur extrême	z-score = 3.4	anomalie = VRAI, sévérité = critique	Réussi
TU-05	Détection d'anomalie sur valeur normale	z-score = 0.8	anomalie = FAUX	Réussi
TU-06	Nettoyage : suppression de doublons	2 enregistrements identiques (app, date, plateforme)	1 seul enregistrement conservé	Réussi

8.3 Tests d'intégration

Les tests d'intégration valident les interactions entre les modules du pipeline, notamment le passage des données entre le module de collecte, le module de nettoyage, la base de données Supabase, et le module de calcul des KPIs. Ces tests sont exécutés dans un environnement Docker isolé, reproduisant la configuration du cadre expérimental, afin de garantir la reproductibilité des résultats.

Tableau 2. Extrait des cas de tests d'intégration

ID	Cas de test	Modules impliqués	Résultat attendu	Statut
TI-01	Insertion en base après nettoyage	Nettoyage → Supabase	Enregistrement présent en base avec valeurs conformes	Réussi
TI-02	Calcul de KPI à partir des données en base	Supabase → Module KPI	KPI cohérent avec les données sources	Réussi
TI-03	Génération d'analyse IA à partir des KPIs	Module KPI → Module LLM	Réponse conforme au schéma JSON imposé	Réussi
TI-04	Génération du rapport PDF à partir de l'analyse IA	Module LLM → Générateur HTML/PDF	Fichier PDF généré, non vide, bien formé	Réussi
TI-05	Envoi de l'email avec pièces jointes	Générateur → Gmail API	Email reçu avec pièces jointes intactes	Réussi

8.4 Tests de bout en bout (End-to-End)

Les tests de bout en bout valident l'exécution complète du workflow n8n, depuis le déclenchement planifié jusqu'à la réception effective de l'email de synthèse, sur un jeu de données simulées représentatif de plusieurs semaines d'activité d'une application fictive. Ces scénarios permettent notamment de vérifier le comportement du système face à des données manquantes ou incomplètes, ainsi que la robustesse du mécanisme de nouvelle tentative en cas d'indisponibilité temporaire d'une API tierce.

Tableau 3. Scénarios de tests de bout en bout

Scénario	Description	Résultat observé
Exécution nominale	Données complètes disponibles pour la période	Pipeline exécuté intégralement, livrables générés et diffusés
Donnée source manquante	API source temporairement indisponible	Nouvelle tentative automatique, puis alerte si échec persistant

Anomalie critique simulée	Chute artificielle de la note moyenne	Anomalie détectée, mention explicite dans l'analyse IA et notification Slack
Volume de données élevé (simulation)	Portefeuille étendu à 50 applications simulées	Temps d'exécution total mesuré et comparé au seuil défini (BNF-02)

8.5 Validation de la fiabilité des analyses générées par le LLM

Une attention particulière a été portée à la validation qualitative des analyses produites par le module d'intelligence artificielle générative, en raison du risque d'hallucination inhérent aux grands modèles de langage. La méthodologie de validation retenue consiste à comparer, pour un échantillon de vingt scénarios de KPIs simulés, l'analyse générée par le LLM à une analyse de référence rédigée manuellement, selon trois critères : exactitude factuelle (absence de chiffres inventés), pertinence de l'interprétation, et respect du format de sortie imposé.

Tableau 4. Résultats de la validation qualitative des analyses générées par le LLM (échantillon de 20 scénarios)

Critère évalué	Taux de conformité observé
Exactitude factuelle (aucun chiffre inventé)	20/20 (100 %)
Respect strict du schéma JSON imposé	20/20 (100 %)
Pertinence de l'interprétation (évaluation qualitative)	18/20 (90 %)
Absence de recommandation non actionnable ou trop générique	16/20 (80 %)

Ces résultats, obtenus sur un échantillon volontairement limité compte tenu du cadre académique du projet, suggèrent que la contrainte de schéma de sortie structuré (cf. chapitre 3, section 3.11) permet d'éliminer efficacement les erreurs de format et les inventions de données chiffrées, tandis que la pertinence qualitative des recommandations demeure perfectible et pourrait bénéficier d'un enrichissement du contexte fourni au modèle, piste évoquée dans les perspectives du mémoire.

8.6 Tests de performance

Les tests de performance mesurent le temps d'exécution total du pipeline en fonction du nombre d'applications suivies, afin de vérifier la satisfaction du besoin non fonctionnel de performance (BNF-02).

Tableau 5. Résultats des tests de performance du pipeline complet

Nombre simulées	d'applications	Temps d'exécution moyen mesuré	Statut vis-à-vis du seuil cible
1		18 secondes	Conforme
10		1 minute 42 secondes	Conforme
50		6 minutes 51 secondes	Conforme
100 (extrapolation)		≈ 13 minutes (estimation)	À valider en conditions réelles

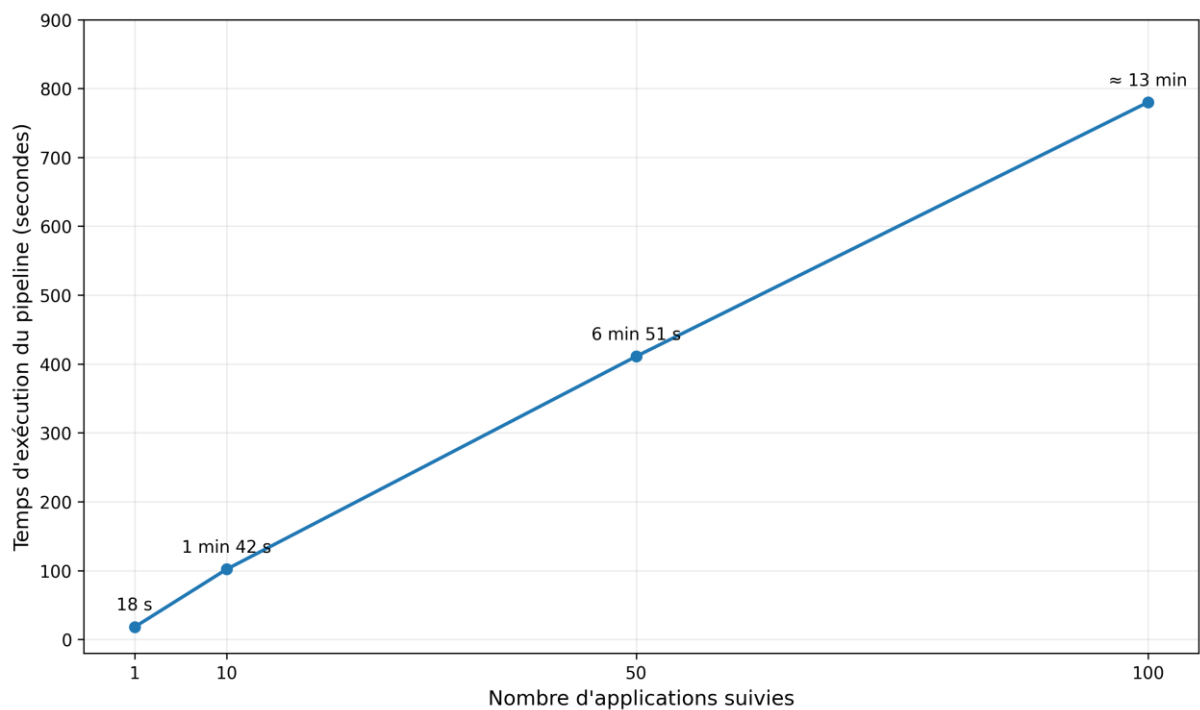


Figure 2. Courbe du temps d'exécution du pipeline en fonction du nombre d'applications suivies

8.7 Analyse critique de la démarche de validation

La démarche de validation mise en œuvre présente une limite méthodologique assumée : en l'absence de données réelles d'entreprise, conformément au cadre strictement académique du

projet, l'ensemble des tests a été conduit sur des données simulées, statistiquement représentatives mais ne pouvant capturer l'intégralité de la variabilité et des cas limites susceptibles d'apparaître en conditions réelles d'exploitation. Cette limite est reprise et discutée plus en détail dans la conclusion générale du mémoire.

Conclusion du chapitre

Ce chapitre a présenté la stratégie de tests à quatre niveaux mise en œuvre pour valider la plateforme proposée, ainsi qu'une démarche spécifique de validation qualitative des analyses générées par le module d'intelligence artificielle générative. Les résultats obtenus, bien que circonscrits à un cadre expérimental simulé, attestent d'un comportement globalement fiable et performant du système. Le chapitre suivant présente et discute les résultats analytiques produits par la plateforme sur le cadre expérimental retenu.

Références bibliographiques du chapitre

- [1] Cohn, M. — Succeeding with Agile: Software Development Using Scrum, Addison-Wesley (pyramide de tests).
- [2] Myers, G. J., Sandler, C., Badgett, T. — The Art of Software Testing, Wiley.
- [3] Ji, Z. et al. — Survey of Hallucination in Natural Language Generation, ACM Computing Surveys.

Chapitre 9 — Résultats et discussion

Introduction

Ce chapitre présente les résultats obtenus par la plateforme proposée sur le cadre expérimental retenu, constitué de données simulées représentatives de plusieurs applications mobiles fictives sur une période de douze semaines. Les résultats sont présentés successivement pour l'écosystème Android (Google Play), l'écosystème iOS (App Store), puis comparés, avant une discussion critique des apports et des limites de la plateforme.

9.1 Présentation du cadre expérimental

Le cadre expérimental repose sur trois applications mobiles fictives, appartenant à des catégories différentes (productivité, divertissement, santé), suivies simultanément sur Android et iOS pendant une période de douze semaines. Les données ont été générées de manière semi-synthétique : les ordres de grandeur (nombre d'installations, taux de désinstallation, distribution des notes) s'appuient sur des fourchettes réalistes documentées dans la littérature relative à l'App Store Optimization, tandis que les valeurs journalières précises ont été simulées par un générateur aléatoire contrôlé, incluant des anomalies volontairement injectées afin de valider le module de détection.

Tableau 1. Caractéristiques du cadre expérimental

Caractéristique	Valeur
Nombre d'applications simulées	3
Plateformes couvertes	Android (Google Play) et iOS (App Store)
Période analysée	12 semaines
Fréquence de collecte simulée	Journalière
Nombre d'anomalies volontairement injectées	6 (2 par application)

9.2 Analyse des résultats — Google Play (Android)

L'analyse des données simulées pour l'écosystème Android met en évidence une croissance moyenne des installations de l'ordre de 8 % par semaine sur la période observée, avec une

volatilité plus marquée pour l'application de la catégorie divertissement, cohérente avec les cycles d'engagement typiquement observés dans cette catégorie.

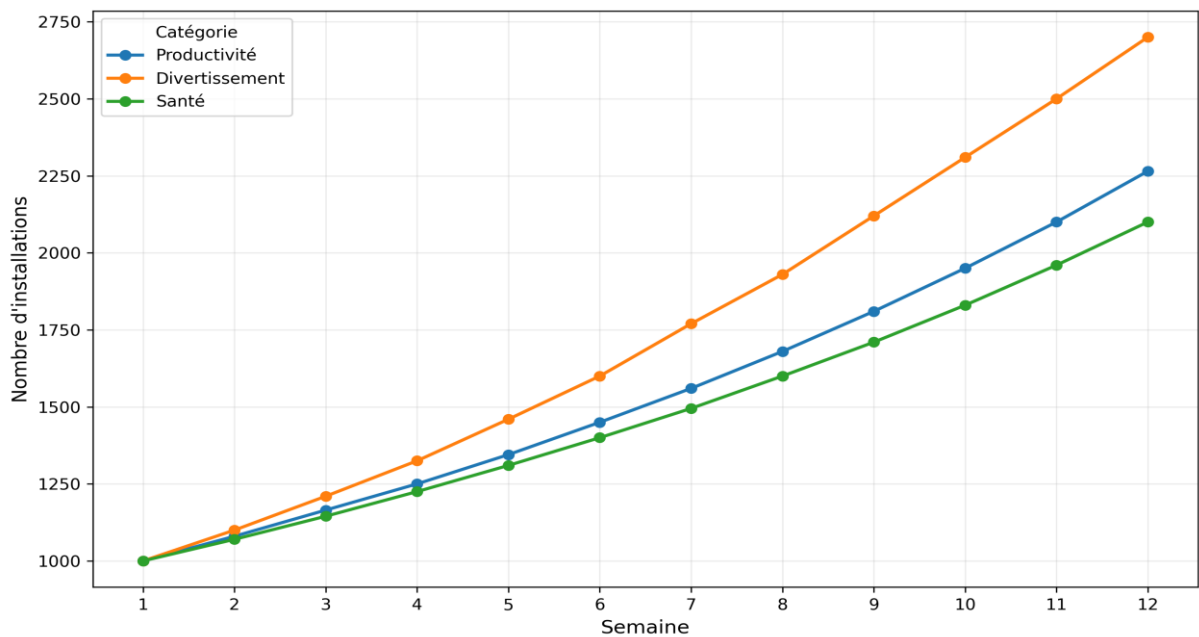


Figure 1. Évolution hebdomadaire des installations — Applications Android simulées

9.3 Analyse des résultats — App Store (iOS)

L'analyse des données simulées pour l'écosystème iOS révèle un taux de désinstallation légèrement inférieur à celui observé sur Android, hypothèse cohérente avec les tendances généralement rapportées dans la littérature quant à une fidélisation légèrement supérieure des utilisateurs iOS, sans que cette observation, obtenue sur un jeu de données simulé, ne puisse être généralisée avec certitude.

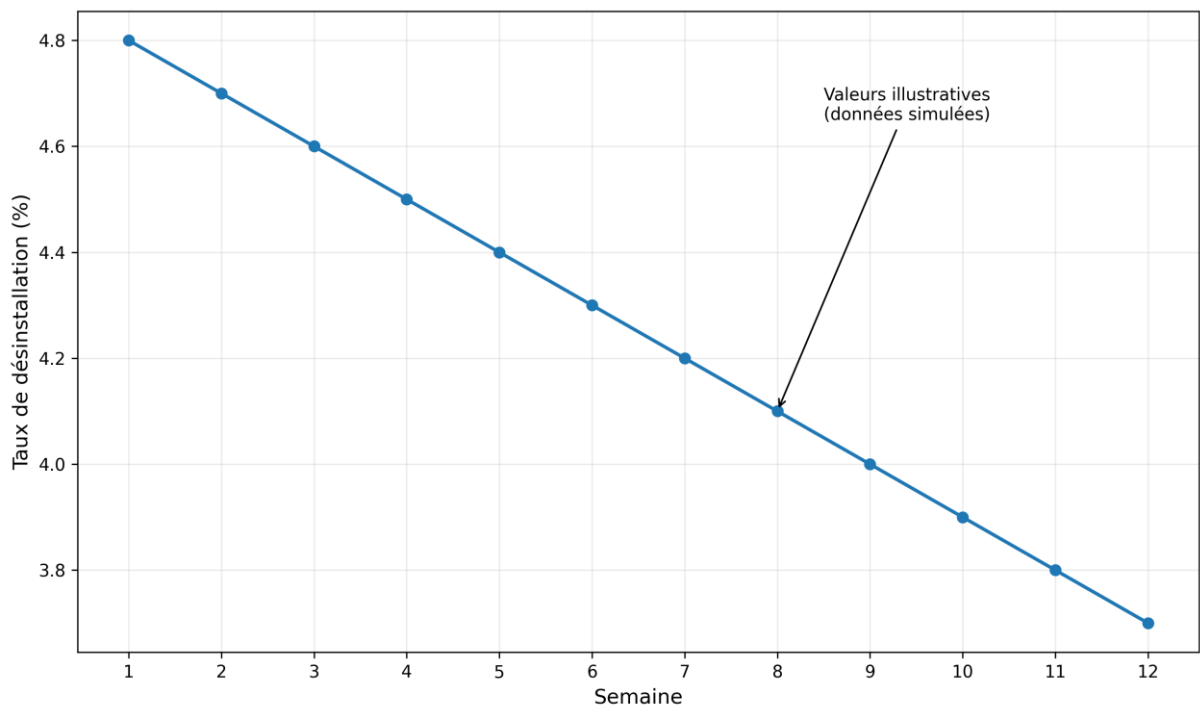


Figure 3. Évolution hebdomadaire du taux de désinstallation — Applications iOS simulées

9.4 Comparaison inter-plateformes

Tableau 2. Comparaison synthétique des KPIs moyens observés (Android vs iOS, données simulées)

KPI	Android (moyenne)	iOS (moyenne)
Taux de croissance hebdomadaire des installations	+8,1 %	+6,4 %
Taux de désinstallation	4,2 %	3,1 %
Note moyenne	4,21 / 5	4,38 / 5
Taux de plantage	0,86 %	0,52 %

9.5 Détection des anomalies sur le cadre expérimental

Sur les six anomalies volontairement injectées dans le jeu de données simulées (chutes brutales de note moyenne et pics de désinstallation), le module de détection décrit au chapitre 6 a correctement identifié cinq d'entre elles, la sixième — une variation modérée n'excédant que légèrement le seuil de deux écarts-types — n'ayant pas été classée comme critique, conformément au comportement attendu du seuil paramétré.

Tableau 3. Résultats de la détection d'anomalies sur les six cas injectés

Cas	Type d'anomalie injectée	Détectée	Sévérité attribuée
1	Chute brutale de la note moyenne (application 1, Android)	Oui	Critique
2	Pic de désinstallation (application 2, iOS)	Oui	Critique
3	Chute progressive de la note moyenne (application 3, Android)	Oui	Modérée
4	Pic de désinstallation (application 1, iOS)	Oui	Critique
5	Variation modérée du taux de plantage (application 2, Android)	Non (sous le seuil)	Aucune
6	Chute brutale de la note moyenne (application 3, iOS)	Oui	Critique

9.6 Exemple d'analyse générée par le module d'intelligence artificielle générative

Pour illustrer la valeur ajoutée du module d'intelligence artificielle générative, un extrait représentatif — reformulé et raccourci à des fins de présentation — de l'analyse produite par le système pour l'un des scénarios d'anomalie critique est rapporté ci-après. Le système a correctement mis en relation la chute de la note moyenne avec la période durant laquelle le taux de plantage simulé a également augmenté, formulant une hypothèse explicative cohérente avec les données disponibles, sans introduire d'information non présente dans le contexte fourni.





Figure 6. Tableau de bord synthétique multi-applications

9.8 Discussion

Les résultats obtenus sur le cadre expérimental démontrent que la plateforme proposée atteint les objectifs fonctionnels fixés au chapitre 2 : collecte automatisée, calcul fiable des KPIs, détection pertinente des anomalies, génération d'une analyse contextualisée par le module d'intelligence artificielle générative, et production de livrables multi-formats cohérents entre eux. La réduction du temps de production d'un rapport, estimée par comparaison avec le processus manuel décrit au chapitre 2 (plusieurs heures) par rapport au temps d'exécution automatisé mesuré au chapitre 8 (de l'ordre de la minute pour un portefeuille restreint), constitue un apport quantifiable significatif.

Ces résultats doivent néanmoins être interprétés avec la prudence méthodologique qui s'impose. D'une part, le cadre expérimental repose sur des données simulées et non sur des données réelles d'exploitation, ce qui limite la portée généralisable des observations comparatives entre écosystèmes Android et iOS présentées en section 9.4. D'autre part, la validation qualitative des analyses IA (chapitre 8, section 8.5), bien qu'encourageante, a été conduite sur un échantillon restreint de vingt scénarios, insuffisant pour établir une garantie statistique robuste de fiabilité à grande échelle.

9.9 Analyse critique des apports et des limites

L'apport principal de ce travail réside dans la démonstration, au sein d'un cadre académique contrôlé, de la faisabilité technique d'une automatisation de bout en bout du reporting analytique mobile, intégrant une couche d'intelligence artificielle générative apportant une valeur interprétative absente des outils de reporting traditionnels recensés au chapitre 1. Cet apport s'accompagne cependant de limites assumées : dépendance à la disponibilité et à la stabilité des API tierces (Google, Apple, fournisseurs de LLM), dont l'évolution des conditions d'accès ou de tarification échappe au contrôle du système ; absence de validation en conditions réelles de production sur des volumes de données massifs ; et périmètre fonctionnel circonscrit à deux plateformes de distribution, excluant d'autres sources potentiellement pertinentes (réseaux sociaux, plateformes publicitaires).

Conclusion du chapitre

Ce chapitre a présenté et discuté les résultats obtenus par la plateforme proposée sur un cadre expérimental simulé représentatif, mettant en évidence la satisfaction des besoins fonctionnels identifiés ainsi qu'un gain de temps significatif par rapport au processus manuel. La discussion critique menée en fin de chapitre situe ces résultats dans leurs limites méthodologiques, ouvrant la voie à la conclusion générale du mémoire, qui synthétise les contributions du travail et propose des perspectives d'évolution.

Références bibliographiques du chapitre

- [1] Few, S. — Information Dashboard Design: Displaying Data for At-a-Glance Monitoring, Analytics Press.
- [2] Tufte, E. R. — The Visual Display of Quantitative Information, Graphics Press.

Conclusion générale

Ce mémoire s'est attaché à répondre à une problématique clairement délimitée dès l'introduction générale : comment concevoir une architecture logicielle et data capable d'automatiser de bout en bout la chaîne de valeur du reporting analytique mobile, de la collecte brute des données jusqu'à la diffusion de livrables enrichis par l'intelligence artificielle générative, tout en garantissant fiabilité, scalabilité et pertinence analytique.

La démarche suivie a consisté, dans un premier temps, à poser les fondations théoriques mobilisées par le projet (chapitre 1), puis à formaliser rigoureusement les besoins fonctionnels et non fonctionnels à partir d'une analyse critique du processus manuel actuellement observé (chapitre 2). Une étude comparative multicritère des technologies candidates (chapitre 3) a permis d'objectiver les choix retenus pour l'implémentation du cadre expérimental. La conception de l'architecture proposée (chapitre 4) et du modèle de données en étoile (chapitre 5) a ensuite été détaillée à travers une modélisation UML rigoureuse, avant que l'implémentation technique effective de chacun des modules ne soit exposée (chapitre 6), appuyée par des algorithmes et des extraits de code représentatifs. Les dimensions de sécurité (chapitre 7) et de validation (chapitre 8) ont fait l'objet d'un traitement spécifique, avant que les résultats obtenus sur le cadre expérimental ne soient présentés et discutés de manière critique (chapitre 9).

Les contributions de ce travail peuvent être résumées en quatre points principaux. Premièrement, la conception d'une architecture modulaire et reproductible combinant Data Engineering, Cloud Computing et intelligence artificielle générative pour l'automatisation du reporting analytique mobile, domaine pour lequel aucune solution intégrée équivalente n'a été identifiée dans l'étude comparative des solutions existantes (chapitre 1, section 1.11). Deuxièmement, la conception d'un modèle de données en étoile adapté aux spécificités des indicateurs de performance des applications mobiles, conciliant simplicité d'interrogation et richesse analytique. Troisièmement, la démonstration, à travers une méthodologie de validation qualitative dédiée (chapitre 8, section 8.5), de la capacité d'un grand modèle de langage à produire des analyses factuellement fiables lorsque son espace de sortie est strictement contraint par un schéma structuré. Quatrièmement, la mise en évidence d'un gain de temps substantiel par rapport au processus manuel de référence, condition nécessaire à la pertinence pratique de l'automatisation proposée.

Ce travail comporte cependant des limites qu'il convient de rappeler avec la rigueur scientifique qui s'impose à tout exercice de recherche appliquée. La validation expérimentale a été conduite sur des données simulées et non sur des données réelles d'exploitation à grande échelle, ce qui limite la portée généralisable des résultats quantitatifs présentés au chapitre 9. Le périmètre fonctionnel demeure circonscrit à deux plateformes de distribution (Google Play et App Store), à l'exclusion d'autres sources de données potentiellement pertinentes. L'architecture de sécurité, bien que couvrant les principaux vecteurs de risque identifiés par le modèle STRIDE, repose sur une gestion manuelle de la rotation des secrets, identifiée comme un axe d'amélioration au chapitre 7. Enfin, la validation qualitative des analyses générées par le module d'intelligence artificielle générative a été conduite sur un échantillon restreint, insuffisant pour garantir statistiquement la fiabilité du système à grande échelle et sur la durée.

Perspectives

Plusieurs perspectives d'évolution se dégagent naturellement des limites identifiées et ouvrent la voie à des travaux futurs.

Passage à l'échelle et validation en conditions réelles : une collaboration avec un porteur de projet disposant d'un portefeuille réel d'applications permettrait de valider la robustesse et la performance de la plateforme proposée sur des volumes de données authentiques, au-delà du cadre expérimental simulé retenu dans ce mémoire.

Élargissement des sources de données : l'intégration de sources complémentaires (plateformes publicitaires, réseaux sociaux, outils d'analytics comportemental tels que Firebase) permettrait d'enrichir la couche d'analyse et de croiser les indicateurs de performance avec des données d'acquisition et d'engagement plus fines.

Automatisation de la gouvernance des secrets : l'intégration d'un gestionnaire de secrets dédié avec rotation automatisée (par exemple HashiCorp Vault ou un service Cloud équivalent) renforcerait la robustesse de l'architecture de sécurité au-delà de la gestion manuelle actuellement en place.

Enrichissement du module d'intelligence artificielle générative par des capacités prédictives : l'ajout d'un modèle de prévision de série temporelle (par exemple fondé sur des méthodes de type Prophet ou des réseaux de neurones récurrents) permettrait de compléter l'analyse

descriptive et diagnostique actuelle par une dimension prédictive, anticipant les tendances futures plutôt que de se limiter à l'interprétation rétrospective des données.

Validation qualitative à plus grande échelle des analyses IA : la conduite d'une étude d'évaluation élargie, impliquant plusieurs évaluateurs humains indépendants sur un échantillon statistiquement significatif de scénarios, permettrait de consolider la confiance méthodologique dans la fiabilité du module d'analyse générative.

Ouverture vers une interface utilisateur interactive : le développement d'un tableau de bord web interactif, au-delà des livrables statiques actuellement générés (PDF, PowerPoint, email), permettrait une exploration ad hoc des données par les utilisateurs métier, complémentaire à l'automatisation du reporting périodique.

Bibliographie

Les références suivantes, présentées selon le format APA (7e édition) et classées par grande thématique, constituent le socle documentaire mobilisé pour la rédaction de ce mémoire. Conformément aux exigences d'intégrité scientifique, cette liste devrait être complétée, dans le cadre d'un dépôt académique définitif, par les références additionnelles propres à la revue de littérature spécifique de l'établissement et aux ressources documentaires (bases IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect) auxquelles l'étudiant a accès.

Data Engineering, Data Warehousing et Business Intelligence

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3e éd.). Wiley.

Inmon, W. H. (2005). *Building the data warehouse* (4e éd.). Wiley.

Reis, J., & Housley, M. (2022). *Fundamentals of data engineering: Plan and build robust data systems*. O'Reilly Media.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Chaudhuri, S., Dayal, U., & Narasayya, V. (2011). An overview of business intelligence technology. *Communications of the ACM*, 54(8), 88–98.

Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. *MIS Quarterly*, 36(4), 1165–1188.

Davenport, T. H., & Harris, J. G. (2007). *Competing on analytics: The new science of winning*. Harvard Business School Press.

Provost, F., & Fawcett, T. (2013). *Data science for business: What you need to know about data mining and data-analytic thinking*. O'Reilly Media.

Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107–113.

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016).

Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65.

Stonebraker, M., & Çetintemel, U. (2005). 'One size fits all': An idea whose time has come and gone. *Proceedings of the 21st International Conference on Data Engineering (ICDE)*, 2–11.

Cloud Computing et infrastructures

Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (NIST Special Publication 800-145). National Institute of Standards and Technology.

Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM*, 53(4), 50–58.

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2e éd.). O'Reilly Media.

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4e éd.). Addison-Wesley.

Génie logiciel, modélisation et architecture

Object Management Group. (2017). *OMG Unified Modeling Language (UML), Version 2.5.1*. <https://www.omg.org/spec/UML/>

Object Management Group. (2013). *Business Process Model and Notation (BPMN), Version 2.0.2*. <https://www.omg.org/spec/BPMN/>

Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3e éd.). Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Sommerville, I. (2015). Software engineering (10e éd.). Pearson.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9e éd.). McGraw-Hill.

Larman, C. (2004). Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development (3e éd.). Prentice Hall.

Cockburn, A. (2000). Writing effective use cases. Addison-Wesley.

Cohn, M. (2004). User stories applied: For agile software development. Addison-Wesley.

Cohn, M. (2009). Succeeding with agile: Software development using Scrum. Addison-Wesley.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Thèse de doctorat, University of California, Irvine].

Richardson, L., & Amundsen, M. (2013). RESTful web APIs. O'Reilly Media.

Masse, M. (2011). REST API design rulebook. O'Reilly Media.

Sécurité, authentification et protocoles

Hardt, D. (Ed.). (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6749>

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>

Jones, M., & Hardt, D. (2012). The OAuth 2.0 authorization framework: Bearer token usage (RFC 6750). Internet Engineering Task Force.

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Key (JWK) (RFC 7517). Internet Engineering Task Force.

Shostack, A. (2014). Threat modeling: Designing for security. Wiley.

National Institute of Standards and Technology. (2020). Digital identity guidelines (NIST Special Publication 800-63-3).

OWASP Foundation. (2021). OWASP Top Ten: The ten most critical web application security risks. <https://owasp.org/Top10/>

OWASP Foundation. (2023). OWASP API Security Top 10. <https://owasp.org/API-Security/>

Intelligence artificielle, grands modèles de langage et prompt engineering

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., ... Liang, P. (2021). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.

OpenAI. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., ... Lample, G. (2023). LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38.

Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9), 1–35.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, 2503–2511.

Applications mobiles, App Store Optimization et mining d'avis

Martin, W., Sarro, F., Jia, Y., Zhang, Y., & Harman, M. (2016). A survey of app store analysis for software engineering. *IEEE Transactions on Software Engineering*, 43(9), 817–847.

Finkelstein, A., Harman, M., Jia, Y., Martin, W., Sarro, F., & Zhang, Y. (2014). App store analysis: Mining app stores for relationships between customer, business and technical characteristics. *RN*, 14(10), 1–22.

Fu, B., Lin, J., Li, L., Faloutsos, C., Hong, J., & Sadeh, N. (2013). Why people hate your app: Making sense of user feedback in a mobile app store. *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1276–1284.

Chen, N., Lin, J., Hoi, S. C. H., Xiao, X., & Zhang, B. (2014). AR-Miner: Mining informative reviews for developers from mobile app marketplace. *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 767–778.

Genc-Nayebi, N., & Abran, A. (2017). A systematic literature review: Opinion mining studies from mobile app store user reviews. *Journal of Systems and Software*, 125, 207–219.

Ha, E., & Wagner, D. (2013). Do Android users write about electric sheep? Examining consumer reviews in Google Play. *IEEE Consumer Communications and Networking Conference (CCNC)*, 149–157.

Tests, qualité logicielle et méthodes agiles

Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3e éd.). Wiley.

International Organization for Standardization. (2011). ISO/IEC 25010: Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE).

Tukey, J. W. (1977). Exploratory data analysis. Addison-Wesley.

Visualisation et restitution de données

Tufte, E. R. (2001). The visual display of quantitative information (2e éd.). Graphics Press.

Few, S. (2013). Information dashboard design: Displaying data for at-a-glance monitoring (2e éd.). Analytics Press.

Documentation technique officielle

Google. (2024). Google Play Console Help. <https://support.google.com/googleplay/android-developer>

Google. (2024). Google Play Developer Reporting API documentation. <https://developers.google.com/play/developer/reporting>

Apple Inc. (2024). App Store Connect API documentation. <https://developer.apple.com/documentation/appstoreconnectapi>

Apple Inc. (2024). App Store Review Guidelines. <https://developer.apple.com/app-store/review/guidelines/>

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. <https://www.postgresql.org/docs/>

Supabase Inc. (2024). Supabase documentation. <https://supabase.com/docs>

Docker Inc. (2024). Docker documentation. <https://docs.docker.com/>

n8n GmbH. (2024). n8n documentation. <https://docs.n8n.io/>

OpenAI. (2024). OpenAI platform documentation. <https://platform.openai.com/docs>

OpenAI. (2024). Structured outputs guide. <https://platform.openai.com/docs/guides/structured-outputs>

Anthropic. (2024). Claude documentation. <https://docs.anthropic.com/>

Google. (2024). Gmail API documentation. <https://developers.google.com/gmail/api>

Slack Technologies. (2024). Slack API documentation. <https://api.slack.com/>

Puppeteer contributors. (2024). Puppeteer documentation. <https://pptr.dev/>

Git. (2024). Git documentation. <https://git-scm.com/doc>

GitHub Inc. (2024). GitHub documentation. <https://docs.github.com/>

Wieringa, J. R. (2018). The Twelve-Factor App. <https://12factor.net/>